

Contents

Technical Design Document: JazzMS Hotel PBX IVR Auto Attendant	3
Table of Contents	3
1. System Overview	3
1.1 High-Level System Description	3
1.2 Design Goals	4
1.3 Constraints	4
1.4 Assumptions	5
1.5 Scope Limitations (Strictly MVP)	5
1.6 MVP Feature Inventory	5
2. Architecture Design	6
2.1 Logical Architecture	6
2.2 Deployment Architecture	10
3. Detailed Call Flow Design	13
3.1 Full Inbound Call Lifecycle	13
3.2 SIP INVITE Flow	15
3.3 Dialplan Match Logic	15
3.4 Lua Script Invocation	15
3.5 DTMF Capture Handling	16
3.6 Timeout Behavior (Feature 1.5, 1.6)	16
3.7 Retry Logic (Features 1.6, 1.7)	16
3.8 Transfer Types	17
3.9 Business Hour Routing (Features 2.1, 3.1)	18
3.10 State Transition Diagram	18
4. Lua IVR Script Design	20
4.1 Script Structure	20
4.2 Session Handling	21
4.3 DTMF Input Collection (Features 1.2, 1.5, 1.6, 1.7)	22
4.4 Prompt Playback (Features 1.3, 1.4, 3.2)	24
4.5 Retry Counter Logic	24
4.6 Failover Routing	25
4.7 Error Handling Strategy	25
4.8 Logging Strategy	25
4.9 Performance Considerations	26
5. Database Design (FusionPBX PostgreSQL)	26
5.1 Schema Strategy	26
5.2 Entity Relationship Description	26
5.3 Table Definitions	29
5.4 Indexing Strategy	33
5.5 Multi-Tenant Isolation via domain_uuid	34
5.6 Migration Strategy	34
6. Prompt & Media Management	35
6.1 Audio Storage Location	35
6.2 TTS Integration (Feature 1.3)	36
6.3 Cache Strategy	37
6.4 Prompt Versioning	37
6.5 File Naming Convention	37

6.6 Audio File Upload (Feature 1.4)	38
6.7 Multi-Language Infrastructure (Feature 3.2)	38
7. Routing Engine Design (Within FusionPBX)	38
7.1 Routing Decision Hierarchy	38
7.2 Domain-Based Routing	39
7.3 Ring Group vs. FIFO Queue Selection	39
7.4 Fallback Routing Strategy	40
7.5 Queue Overflow Logic	40
7.6 Voicemail Fallback (Feature 2.2)	40
7.7 Implementation Across FusionPBX Components	41
8. Reporting & Metrics	42
8.1 CDR Extraction (Feature 5.1)	42
8.2 Data Aggregation Approach	42
8.3 Reporting Queries	43
8.4 Real-Time Metrics Dashboard (Feature 5.2)	44
9. Security & Compliance	45
9.1 SIP Security (Feature 6.1)	45
9.2 PostgreSQL Access Control	46
9.3 FusionPBX RBAC Roles	47
9.4 API Authentication	47
9.5 Audit Logging	48
9.6 Tenant Isolation	48
9.7 Firewall and Fail2Ban	48
10. Performance & Scalability	49
10.1 Expected Concurrent Call Handling	49
10.2 FreeSWITCH Performance Tuning	50
10.3 CPU/Memory Sizing	50
10.4 PostgreSQL Optimization	50
10.5 Connection Pooling	51
10.6 Lua Script Performance Considerations	51
10.7 CDR Indexing Strategy	52
11. Failure Handling & Resilience	52
11.1 FreeSWITCH Crash Recovery	52
11.2 SIP Trunk Failure Handling	53
11.3 Database Failover Handling	53
11.4 Dialplan Fallback Logic	54
11.5 Retry Mechanisms Summary	54
11.6 Dead Route Handling	55
11.7 Media Server Overload Protection	55
12. Observability & Monitoring	55
12.1 FreeSWITCH Logs	55
12.2 FusionPBX Logs	56
12.3 PostgreSQL Logs	56
12.4 Call Metrics Monitoring	56
12.5 SIP Health Checks	57
12.6 Alerting Strategy	57
12.7 Log Retention Policy	58
13. Deployment & DevOps Strategy	58

13.1 Environment Setup	58
13.2 CI/CD for Lua Scripts	58
13.3 Version Control Strategy	59
13.4 Blue-Green Deployment	59
13.5 Database Migration Handling	59
13.6 Backup & Restore Plan	60
14. Technology Stack	60
14.1 Fixed Requirements	60
14.2 Configuration Decisions and Tuning Parameters	62
Appendix A: Glossary	64
Appendix B: MVP Feature-to-Section Traceability Matrix	65

Technical Design Document: JazzMS Hotel PBX IVR Auto Attendant

Product: JazzMS IVR Auto Attendant for Hotel PBX **Phase:** Phase 1 — Minimum Viable Product (MVP) **Platform:** FusionPBX / FreeSWITCH / PostgreSQL / Lua **Classification:** Carrier-Grade, Multi-Tenant **Version:** 1.0 **Date:** February 19, 2026

Table of Contents

1. [System Overview](#)
 2. [Architecture Design](#)
 3. [Detailed Call Flow Design](#)
 4. [Lua IVR Script Design](#)
 5. [Database Design \(FusionPBX PostgreSQL\)](#)
 6. [Prompt & Media Management](#)
 7. [Routing Engine Design](#)
 8. [Reporting & Metrics](#)
 9. [Security & Compliance](#)
 10. [Performance & Scalability](#)
 11. [Failure Handling & Resilience](#)
 12. [Observability & Monitoring](#)
 13. [Deployment & DevOps Strategy](#)
 14. [Technology Stack](#)
-

1. System Overview

1.1 High-Level System Description

JazzMS IVR Auto Attendant is a cloud-hosted, carrier-grade Interactive Voice Response system purpose-built for the hospitality industry. It operates within the FusionPBX telephony platform, leveraging FreeSWITCH as the underlying media and call-control engine. The system automates inbound call routing for hotels and multi-property chains,

reducing front desk workload by 30–40% while providing a professional guest experience 24/7.

The IVR receives inbound SIP calls on configured trunks, presents callers with multi-level DTMF-driven menus, validates input, applies business-hour and time-based routing rules, integrates with the hotel's Property Management System (PMS) for room-level validation, and delivers calls to the appropriate department queue, extension, voicemail box, or room phone. All call activity is logged for analytics and reporting.

1.2 Design Goals

#	Goal	Measure
G1	Carrier-grade reliability	99.99% uptime; zero dropped calls due to software faults
G2	Low latency call processing	< 200 ms from DTMF receipt to routing action
G3	Multi-tenant isolation	Strict per-domain data separation via FusionPBX domain_uuid
G4	Horizontal scalability	Support 500+ concurrent calls per node; cluster for more
G5	Operational simplicity	All configuration via FusionPBX admin UI or database; no code deploys for menu changes
G6	Security by default	TLS 1.2+ signaling, mandatory SRTP media, encrypted database connections
G7	MVP scope discipline	Ship only the 18 features defined in the Phase 1 feature list

1.3 Constraints

Constraint	Description
C1	All call processing MUST execute within FusionPBX dialplan and Lua scripting; no external application servers for call logic.
C2	Database MUST be the FusionPBX default PostgreSQL instance (version 15+). Custom tables extend but never modify the default schema.
C3	IVR scripting MUST use Lua via FreeSWITCH mod_lua. No Python, JavaScript, or external runtimes in the call path.
C4	SIP connectivity MUST use mod_sofia profiles already managed by FusionPBX.

Constraint	Description
C5	Audio prompts stored on the FusionPBX filesystem under the standard recordings directory structure.
C6	Multi-language support is infrastructure-ready but English-only for MVP.
C7	No VIP priority queuing, callback queues, holiday-specific closures, emergency routing, wake-up calls, or channel handoff (SMS/chat) — all deferred to Phase 2.

1.4 Assumptions

1. Each hotel property maps to one FusionPBX domain (tenant).
2. Hotel PBX endpoints (room phones, agent phones) are reachable via SIP from the FusionPBX node — either directly registered or via a SIP trunk to an on-premise PBX.
3. Each property has a PMS that exposes a REST API for room status, DND, and occupancy queries.
4. TTS is provided by an external cloud API (Google Cloud TTS or AWS Polly); network connectivity from the FusionPBX node to the TTS endpoint is available.
5. Voicemail audio files can be stored either on local disk or S3-compatible object storage; email delivery via SMTP is available.
6. Administrative users manage IVR menus, prompts, and schedules through FusionPBX's web UI or direct database configuration.

1.5 Scope Limitations (Strictly MVP)

The following capabilities are **explicitly excluded** from this design and deferred to Phase 2:

- Holiday calendar routing (ad-hoc emergency closures)
- VIP caller detection and priority queuing
- Natural language / speech recognition input
- Multi-language prompts beyond English
- Restaurant / spa / amenity booking via IVR
- Housekeeping self-service menus
- Wake-up call scheduling
- Virtual hold / callback queue
- SMS or web-chat handoff
- Call parking (optional mention in MVP spec; excluded from TDD)

1.6 MVP Feature Inventory

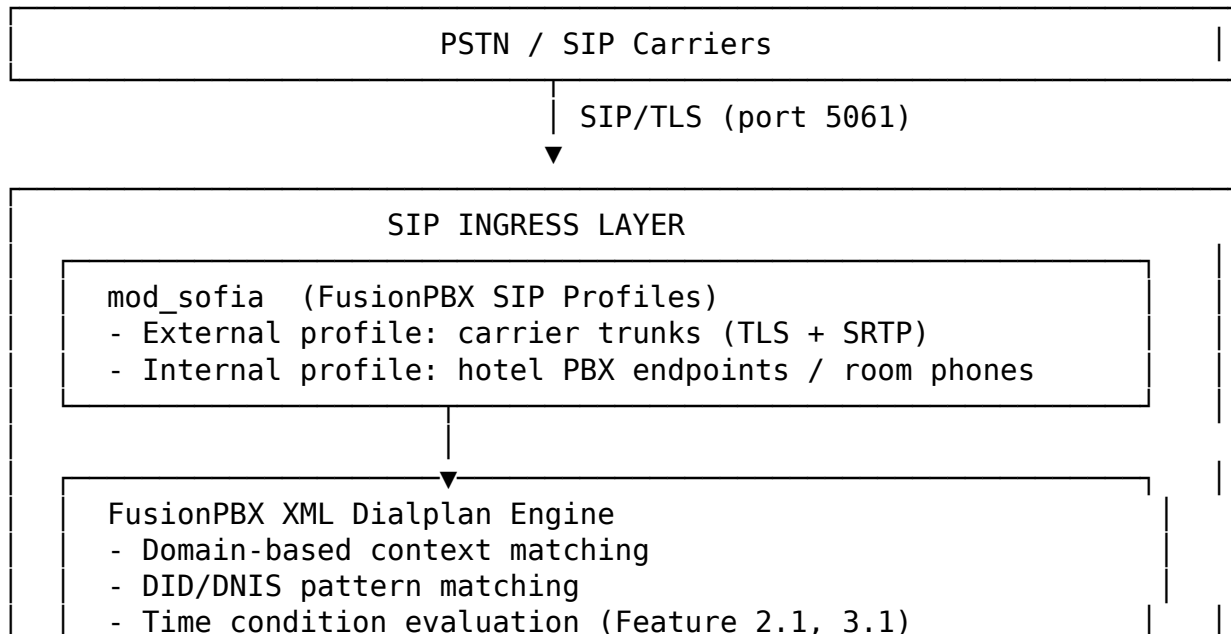
ID	Feature	Priority	Classification
1.1	Multi-Level IVR Navigation	P0	Must Have
1.2	DTMF Input Capture & Validation	P0	Must Have
1.3	Text-to-Speech (TTS) Integration	P1	Must Have
1.4	Audio File Upload & Management	P1	Must Have
1.5	Configurable Timeouts & Retries	P1	Must Have
1.6	No-Input Timeout Handling	P1	Must Have
1.7	Invalid Input Handling	P1	Must Have
2.1	Time-Based Routing	P1	Should Have
2.2	Voicemail Routing & Fallback	P1	Must Have
2.3	Room Extension Routing	P1	Must Have
3.1	Business Hours Calendar	P1	Should Have
3.2	Language Selection (English MVP)	P1	Must Have
4.1	PMS Integration	P1	Must Have
4.2	Hotel PBX Integration	P1	Must Have
5.1	Call Detail Records (CDR)	P1	Must Have
5.2	Real-Time Metrics Dashboard	P1	Should Have
6.1	Encryption (TLS + SRTP)	P0	Must Have

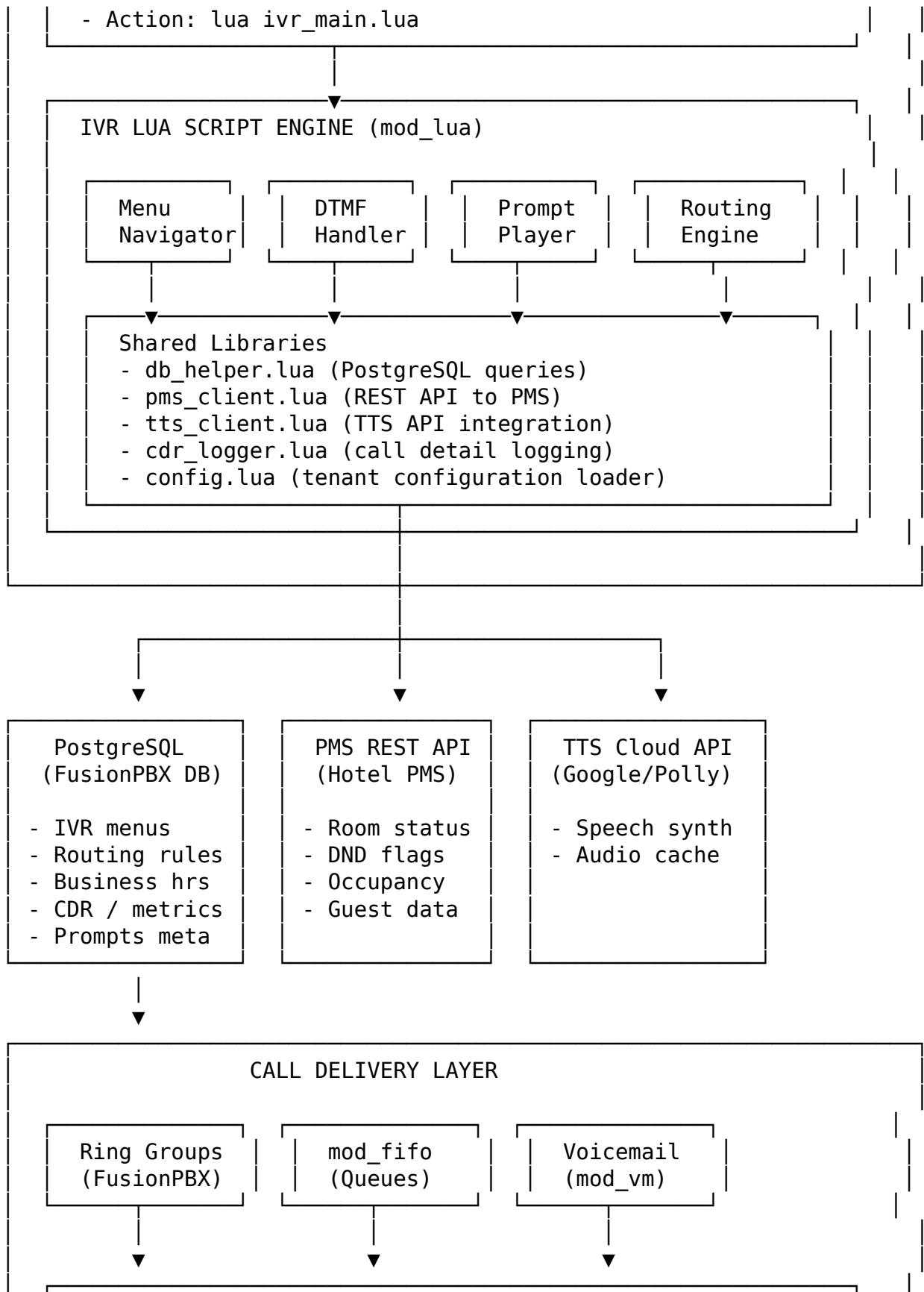
Totals: 18 features — 3 P0 Must Have, 12 P1 Must Have, 3 P1 Should Have.

2. Architecture Design

2.1 Logical Architecture

2.1.1 Component Overview





Hotel PBX Endpoints	
- Agent desk phones (SIP)	- Room phones (SIP)
- Softphones	- Mobile via SIP trunk

2.1.2 SIP Ingress (Trunk → FusionPBX) Inbound calls arrive from PSTN carriers or SIP trunking providers on the **external** `mod_sofia` profile. FusionPBX manages this profile with the following characteristics:

- **Transport:** SIP over TLS (port 5061); UDP/TCP (port 5060) disabled in production per Feature 6.1.
- **Media:** SRTP mandatory; `crypto:AES_CM_128_HMAC_SHA1_80` required.
- **Codecs:** G.711 μ -law (PCMU), G.711 a-law (PCMA), G.729 — negotiated in SDP.
- **DTMF:** RFC 2833 / RFC 4733 preferred; in-band detection as fallback via `mod_spandsp`.
- **Caller identification:** ANI extracted from SIP From / P-Asserted-Identity; DNIS from To / Request-URI.

On INVITE receipt, `mod_sofia` authenticates the trunk (IP ACL + optional digest auth), applies the FusionPBX domain context based on the DID match, and passes control to the XML dialplan.

2.1.3 Dialplan Processing FusionPBX stores dialplan entries in PostgreSQL and renders them as XML contexts at runtime. The IVR call flow uses the following dialplan chain:

1. **Public context** — matches the inbound DID number against the tenant's configured destinations.
2. **DID routing** — maps the DID to the IVR application. The destination action is:

```
<action application="lua" data="ivr_main.lua ${domain_name} ${destination_number}" />
```
3. **Time conditions** (Feature 2.1, 3.1) — evaluated inside the Lua script (not XML time conditions) for per-department granularity and database-driven schedule management.
4. **Transfer targets** — when the Lua script completes routing decisions, it issues `session:transfer()` to FusionPBX ring groups, FIFO queues, extensions, or voicemail.

2.1.4 IVR Lua Script Engine The core IVR logic executes inside FreeSWITCH's `mod_lua` environment. A single entry-point script (`ivr_main.lua`) bootstraps the call session, loads tenant configuration from PostgreSQL, and delegates to modular handlers:

- **Menu Navigator** — traverses the `ivr_menus` table tree; tracks breadcrumb path; handles * (back) and # (repeat).

- **DTMF Handler** — wraps `session:playAndGetDigits()` with configurable timeouts, inter-digit timers, and retry counters (Features 1.2, 1.5, 1.6, 1.7).
- **Prompt Player** — resolves prompt source (audio file vs. TTS), invokes TTS client if needed, caches result, and calls `session:streamFile()` (Features 1.3, 1.4, 3.2).
- **Routing Engine** — evaluates business hours, determines target (queue, extension, room, voicemail), and executes the transfer (Features 2.1, 2.2, 2.3).

2.1.5 Ring Groups and Queues For department-level call delivery, the system uses FusionPBX's native constructs:

- **Ring Groups** — used for small, fixed teams (e.g., front desk with 3 agents). Configured with ring strategy (simultaneous, sequential, random) and timeout. Built-in FusionPBX ring group application.
- **mod_fifo** — used for queue-based routing where calls must wait for the next available agent. Provides hold music, position announcements, and queue timeout with fallback. Chosen over `mod_callcenter` for MVP simplicity; `mod_fifo` has lower configuration overhead, integrates natively with FusionPBX, and meets the MVP requirement of basic queue behavior without agent-tier or skill-based routing (deferred to Phase 2).

Voicemail is handled by FusionPBX's built-in voicemail application (`mod_voicemail`), configured per-extension and per-department with custom greetings, max recording length (3 minutes per Feature 2.2), and email notification via SMTP.

2.1.6 Reporting and Admin

- **CDR** (Feature 5.1) — the Lua script writes enriched CDR records (navigation path, DTMF sequence, disposition) to the `ivr_cdr` custom table at call completion. FusionPBX's native `xml_cdr` table provides the base SIP-level CDR.
- **Real-Time Metrics** (Feature 5.2) — a lightweight polling API reads from `ivr_cdr` and FreeSWITCH ESL (`mod_event_socket`) to aggregate live counters. Dashboard served as a FusionPBX app page or standalone web view.
- **Admin Configuration** — IVR menus, routing rules, business hours, and prompt metadata are stored in PostgreSQL and managed via FusionPBX superadmin/domain-admin UI pages.

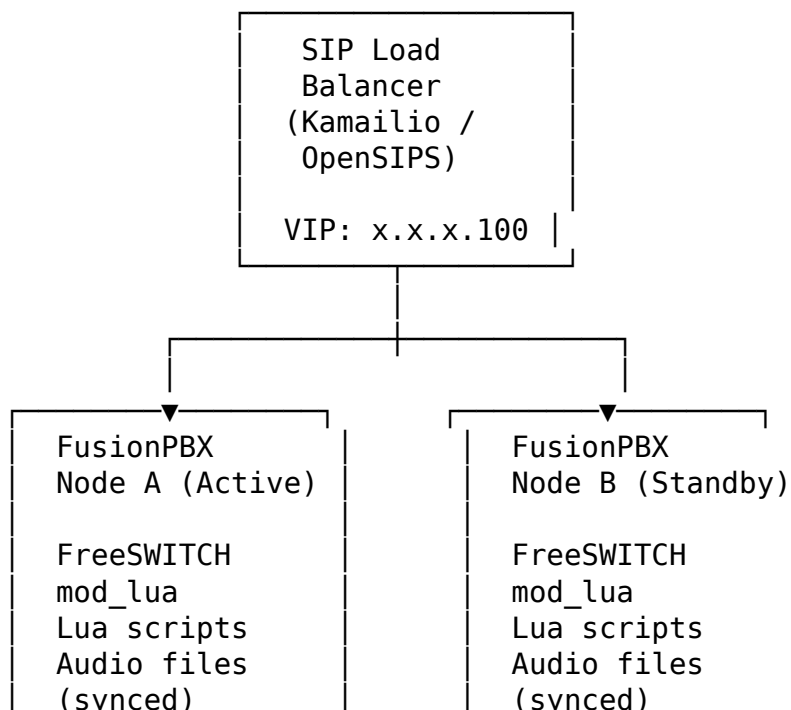
2.1.7 FusionPBX Component Interaction Map

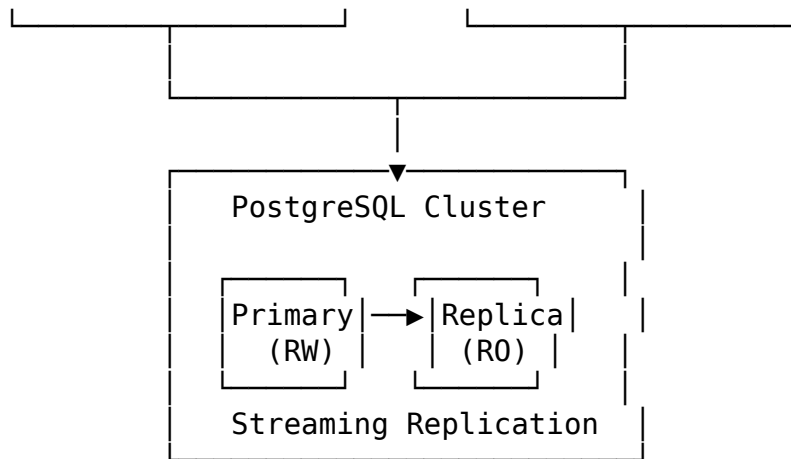
Component	Role in IVR System
XML Dialplan	Routes inbound DID to the Lua IVR script; provides extension-level routing for transfers.
mod_lua	Executes all IVR business logic — menu traversal, DTMF collection, timeout/retry handling, PMS queries, TTS invocation, CDR logging.

Component	Role in IVR System
mod_sofia	Manages SIP profiles for external trunks (carrier ingress, TLS) and internal endpoints (hotel PBX, room phones). Handles SIP INVITE/BYE/REFER for call setup and transfer.
PostgreSQL	Stores tenant configuration, IVR menu trees, routing rules, business hours, prompt metadata, CDR records, and real-time metric aggregates. Extends FusionPBX's default schema with custom tables.
mod_fifo	Provides FIFO-based call queuing for departments. Callers wait with hold music; agents are dispatched on availability.
mod_voicemail	Records voicemail messages, stores audio, sends email notifications with audio attachments, manages MWI (Message Waiting Indicator) on SIP phones.
mod_local_stream / mod_shout	Streams hold music and prompt audio files.
mod_spandsp	In-band DTMF detection fallback.
mod_event_socket (ESL)	Provides real-time call event data for the metrics dashboard.

2.2 Deployment Architecture

2.2.1 Topology: Active-Standby Clustered FusionPBX For carrier-grade availability, the MVP deploys a **two-node active-standby cluster** with a shared database tier.





2.2.2 High Availability Strategy

Component	HA Mechanism	Details
SIP Ingress	SIP-aware load balancer (Kamailio or OpenSIPS)	Distributes SIP INVITE to the active node. Health-checks via SIP OPTIONS pings every 5 seconds. Failover < 10 seconds.
Floating VIP	Keepalived + VRRP	The SIP LB binds to a floating VIP. If the primary LB fails, the standby LB assumes the VIP within 3 seconds.
FusionPBX Nodes	Active-Standby with shared DB	Node A handles all traffic. Node B is warm-standby — FreeSWITCH running, Lua scripts deployed, ready to accept calls if LB redirects.
PostgreSQL	Streaming replication (async)	Primary → Replica with < 1 second lag. Automatic promotion via Patroni or pg_auto_failover. RPO < 1 second; RTO < 30 seconds.

Component	HA Mechanism	Details
Media / Audio Files	lsyncd or rsync cron	Lua scripts and audio files synchronized from Node A → Node B every 60 seconds. Git-based deployment ensures identical versions.
SIP Trunks	Dual trunk registration	Carriers register to both nodes; LB routes based on active node. Alternatively, carrier-side failover to secondary IP.

2.2.3 Database Replication Strategy PostgreSQL streaming replication is configured as follows:

- **Primary** (read-write): Receives all INSERT/UPDATE from Lua scripts and FusionPBX admin UI.
- **Replica** (read-only): Synchronizes via WAL streaming. Used for read-heavy reporting queries (CDR, metrics) to offload the primary.
- **Failover orchestrator**: Patroni manages automatic leader election if the primary fails. The FusionPBX database connection string uses a Patroni-managed endpoint or PgBouncer that resolves to the current primary.
- **Backup**: Daily pg_basebackup to S3-compatible storage with 30-day retention. WAL archiving to S3 for point-in-time recovery.

2.2.4 Node Specifications

Node Role	vCPUs	RAM	Storage	OS
FusionPBX + FreeSWITCH (per node)	8	16 GB	100 GB SSD (OS + recordings)	Ubuntu 22.04 LTS
PostgreSQL Primary	4	8 GB	200 GB SSD (data + WAL)	Ubuntu 22.04 LTS
PostgreSQL Replica	4	8 GB	200 GB SSD	Ubuntu 22.04 LTS
SIP Load Balancer	2	4 GB	20 GB SSD	Ubuntu 22.04 LTS

2.2.5 Storage Layout

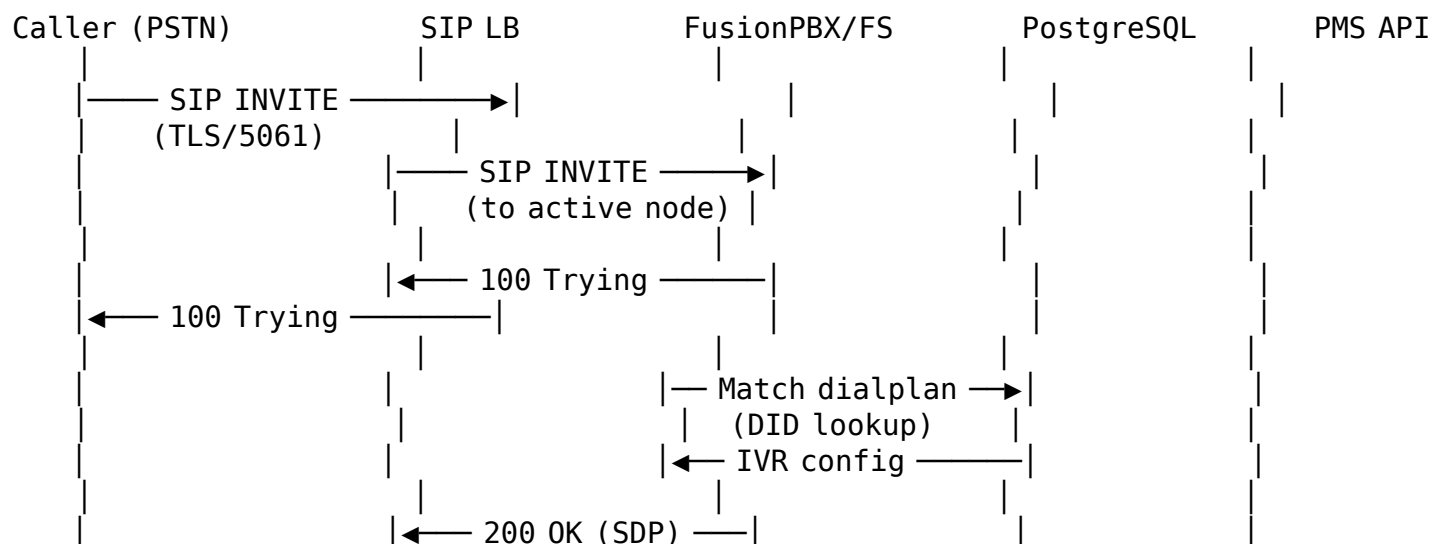
Path	Purpose
/var/lib/freeswitch/recordings/Audio prompts	Audio prompts per tenant
/var/lib/freeswitch/recordings/Voice mail recordings	Voice mail recordings
/usr/share/freeswitch/scripts/Lua	IVR scripts
/var/log/freeswitch/	FreeSWITCH logs
/var/log/fusionpbx/	FusionPBX application logs
/var/lib/freeswitch/tts_cache/	TTS audio cache files

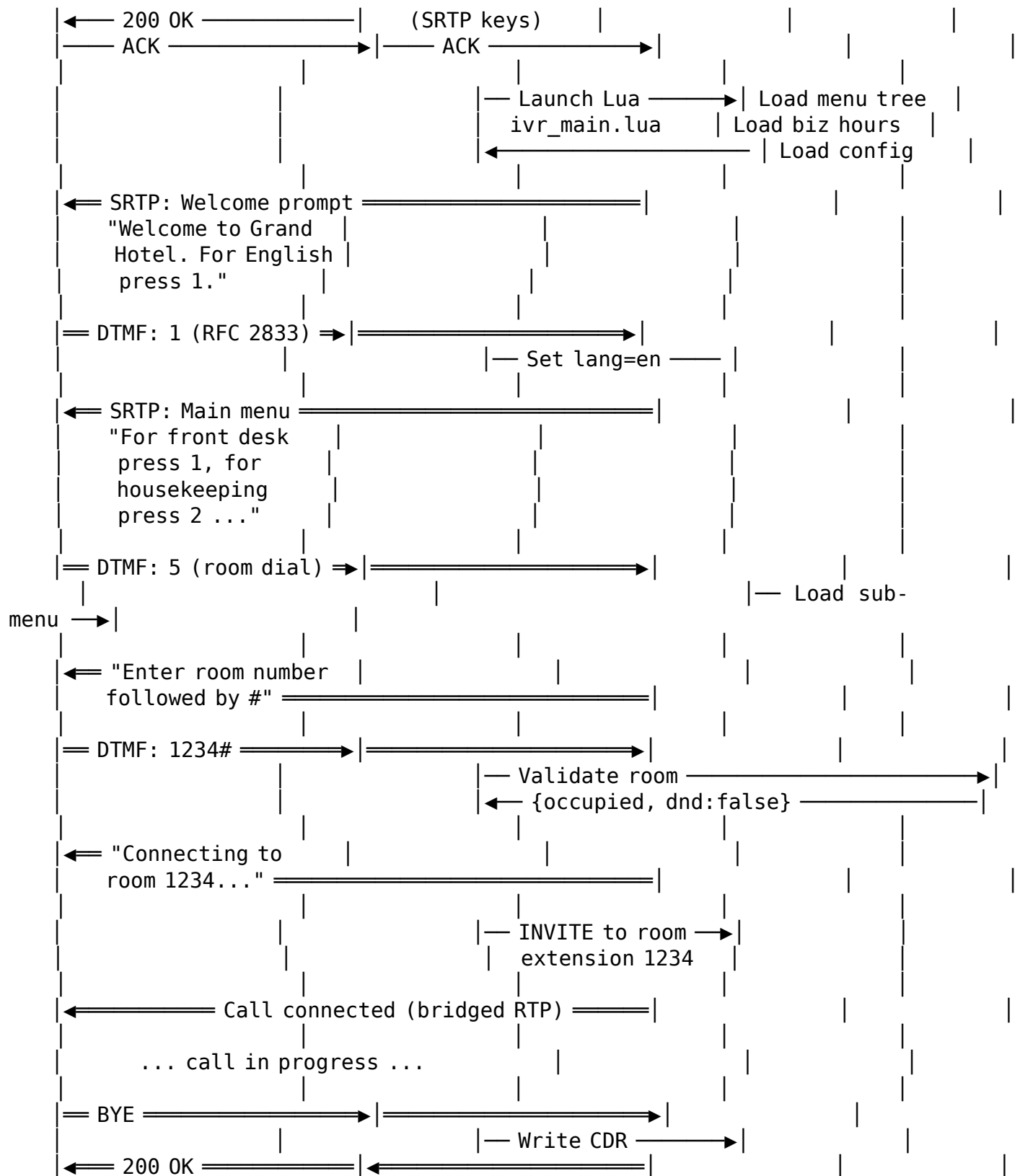
2.2.6 Backup Strategy

Data	Method	Frequency	Retention
PostgreSQL (full)	pg_basebackup → S3	Daily 02:00 UTC	30 days
PostgreSQL (WAL)	Continuous archiving → S3	Continuous	7 days
Lua scripts	Git repository	On every deploy	Indefinite (Git history)
Audio prompts	rsync → S3	Daily 03:00 UTC	90 days
Voicemail recordings	rsync → S3	Hourly	90 days (configurable)
FusionPBX configuration	FusionPBX backup module → S3	Daily 04:00 UTC	30 days

3. Detailed Call Flow Design

3.1 Full Inbound Call Lifecycle





3.2 SIP INVITE Flow

1. **Carrier → SIP LB:** SIP INVITE arrives on port 5061 (TLS). The SIP load balancer inspects the Request-URI, applies rate limiting, and forwards to the active FusionPBX node.
2. **SIP LB → FusionPBX:** INVITE forwarded. mod_sofia external profile accepts the call. The From header provides ANI; To / Request-URI provides DNIS.
3. **FusionPBX Dialplan Match:** The public context matches the DNIS against v_dialplan_details where the destination is configured as an IVR application.
4. **Early Media:** A 183 Session Progress with SDP is sent if ringback is needed. For IVR, the call is answered immediately (200 OK) to begin prompt playback.
5. **SRTP Negotiation:** SDP includes a=crypto lines for SDES key exchange. Both sides agree on AES_CM_128_HMAC_SHA1_80.
6. **ACK:** Caller confirms; RTP/RTSP media path is established.

3.3 Dialplan Match Logic

The FusionPBX dialplan for IVR calls follows this structure:

```
<!-- Public context: inbound DID routing -->
<extension name="hotel_ivr_inbound" continue="false">
  <condition field="destination_number" expression="^\(+1555010[0-9])$">
    <action application="set" data="domain_name=grandhotel.example.com"/>
    <action application="set" data="domain_uuid=alb2c3d4-..."/>
    <action application="set" data="call_direction=inbound"/>
    <action application="set" data="hangup_after_bridge=true"/>
    <action application="set" data="ivr_entry_did=${destination_number}"/>
    <action application="answer"/>
    <action application="sleep" data="500"/>
    <action application="lua" data="ivr/ivr_main.lua"/>
  </condition>
</extension>
```

Key aspects: - domain_name and domain_uuid are set for multi-tenant isolation — all subsequent Lua database queries filter on domain_uuid. - The call is answered before Lua execution to enable immediate prompt playback. - A 500 ms sleep prevents audio clipping on call setup.

3.4 Lua Script Invocation

When the dialplan executes application="lua" data="ivr/ivr_main.lua", FreeSWITCH spawns a Lua coroutine within mod_lua. The script receives the session object representing the active call. The script:

1. Reads channel variables (domain_uuid, caller_id_number, destination_number, ivr_entry_did).
2. Opens a PostgreSQL connection via freeswitch.Dbh (connection pooled).
3. Loads the tenant's IVR menu tree, business hours, and configuration.
4. Enters the menu navigation loop.

3.5 DTMF Capture Handling

DTMF is captured using `session:playAndGetDigits()`:

```
session:playAndGetDigits(
    min_digits,          -- 1 for menu selection; 3-6 for room number
    max_digits,          -- 1 for menu selection; 6 for room number
    max_attempts,        -- from config (default 3)
    timeout_ms,          -- initial timeout (default 5000ms)
    terminators,        -- "#" for multi-digit; "" for single-digit
    prompt_file,         -- audio file path or TTS-generated file
    error_file,          -- invalid input prompt
    digit_regex,         -- regex to validate input (e.g., "[1-5]" for a 5-
option menu)
    variable_name,       -- channel variable to store result
    inter_digit_ms       -- inter-digit timeout (default 3000ms)
)
```

Barge-in: Enabled by default — when the caller presses a valid DTMF key during prompt playback, the prompt stops immediately and the digit is processed.

3.6 Timeout Behavior (Feature 1.5, 1.6)

Each menu level maintains two independent counters:

- `no_input_counter` — incremented when `playAndGetDigits` returns empty string (timeout with no DTMF).
- `invalid_input_counter` — incremented when the returned digit(s) don't match any valid menu option.

Both counters are configured with: - **Default max retries:** 3 (configurable 1-5 per menu) - **Initial timeout:** 5000 ms (configurable 3000-10000 ms per menu) - **Retry timeout:** 5000 ms (configurable 3000-10000 ms per menu)

Counter reset rule: Both counters reset to 0 when the caller provides a valid input.

3.7 Retry Logic (Features 1.6, 1.7)

No-Input Progressive Prompts:

Attempt	Prompt	Action After Prompt
1	"I didn't receive your selection. Please try again."	Replay menu
2	"I still haven't received a selection. Please press a number now."	Replay menu

Attempt	Prompt	Action After Prompt
3 (final)	"I'm not receiving any input. Let me transfer you to an agent."	Route to operator/fallback

Invalid-Input Progressive Prompts:

Attempt	Prompt	Action After Prompt
1	"That is not a valid option. Please try again."	Replay menu
2	"I'm sorry, that selection is not available. Please listen carefully."	Replay menu
3 (final)	"I'm having trouble with your selection. Press 0 for an operator or * to hear the menu again."	Offer escape hatch; then route to operator if no valid input

3.8 Transfer Types

3.8.1 Blind Transfer Used for department queue routing and room extensions:

```
session:transfer(destination, "XML", domain_name)
```

The Lua script sets appropriate channel variables and transfers the call to the FusionPBX ring group, FIFO queue, or extension. The original IVR session ends; the call proceeds directly.

3.8.2 Attended Transfer Used when the Lua script must verify destination availability before connecting (e.g., room phone DND check):

```
session:execute("bridge", "sofia/internal/" .. extension .. "@" .. domain)
```

The bridge application connects the caller to the destination. If the bridge fails (no answer, busy, rejected), control returns to Lua for fallback handling (voicemail option, retry, main menu).

3.8.3 Queue Behavior (mod_fifo) When routing to a department queue: 1. Lua script transfers the call to the FIFO queue name (e.g., front_desk_fifo@domain). 2.

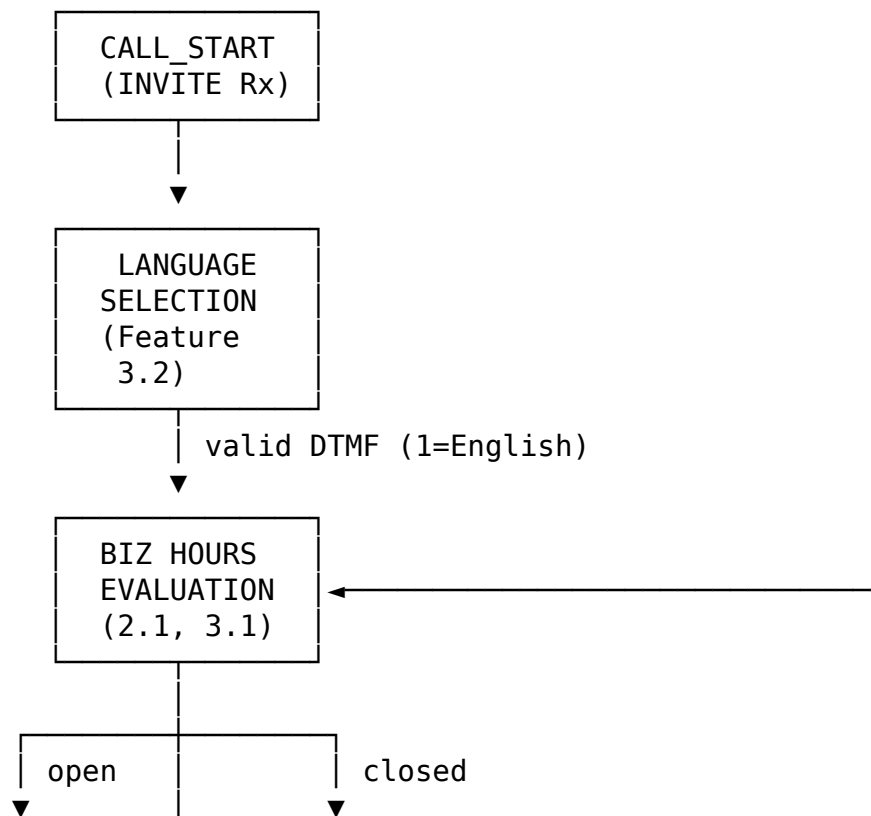
mod_fifo places the caller in the queue, plays hold music. 3. Agents are members of the FIFO and receive calls in configured order. 4. If the queue timeout expires (configurable, default 45 seconds for MVP), mod_fifo executes the timeout action — the dialplan routes back to Lua for voicemail fallback (Feature 2.2).

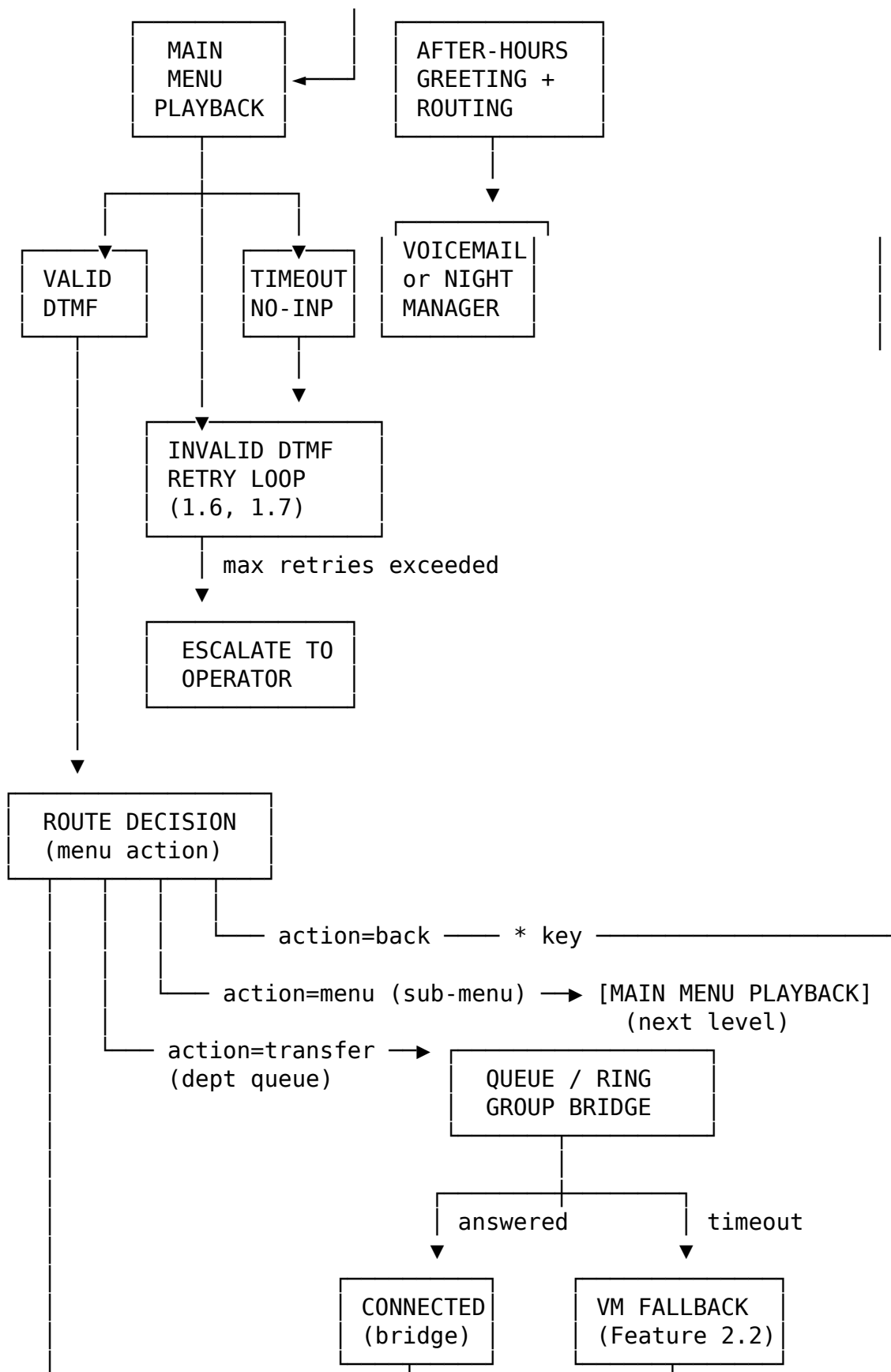
3.9 Business Hour Routing (Features 2.1, 3.1)

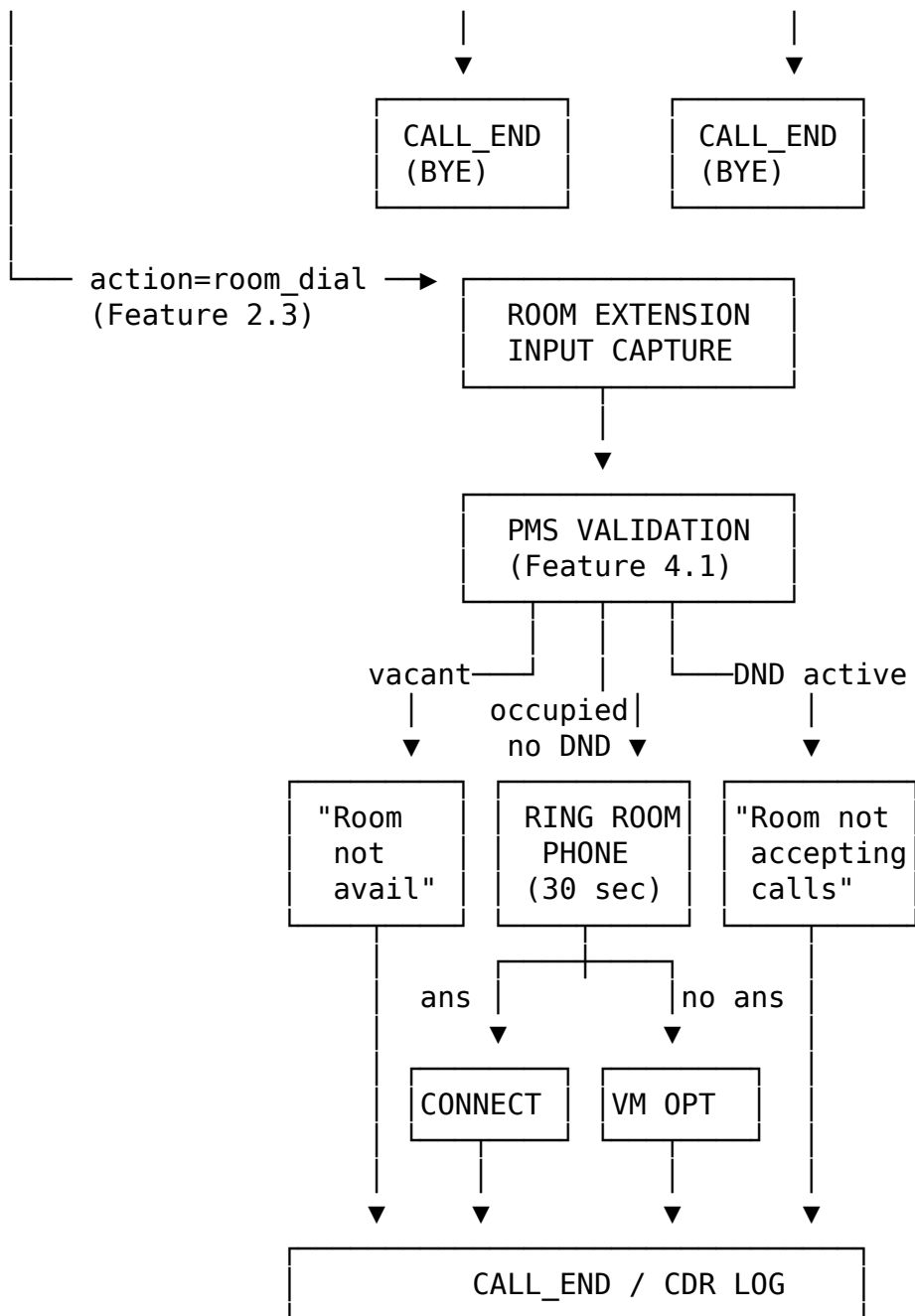
Time-based routing is evaluated in the Lua script at the start of each call:

1. Load the department's business_hours record from PostgreSQL, including time-zone.
2. Convert current UTC time to the hotel's local timezone using `os.date()` with time-zone offset.
3. Determine the current day-of-week and time.
4. Compare against the department's open/close hours for that day.
5. Select the appropriate routing target:
 - **Within business hours:** Route to the department's queue/ring group with the business-hours greeting.
 - **After hours:** Route to the after-hours destination (night manager mobile, voicemail) with the after-hours greeting.
6. Check for manual override flag — if set, use override routing regardless of schedule.

3.10 State Transition Diagram







4. Lua IVR Script Design

4.1 Script Structure

```

/usr/share/freeswitch/scripts/ivr/
├── ivr_main.lua           -- Entry point; session bootstrap
├── lib/
│   └── menu_navigator.lua -- Menu tree traversal and breadcrumb tracking

```

```

├── dtmf_handler.lua      -- DTMF collection with timeout/retry logic
├── prompt_player.lua     -- Audio file / TTS playback with caching
├── routing_engine.lua    -- Business hours, transfers, queue routing
├── room_router.lua       -- Room extension routing with PMS validation
├── voicemail_handler.lua -- Voicemail fallback logic
├── cdr_logger.lua        -- Enriched CDR writing to PostgreSQL
├── db_helper.lua        -- PostgreSQL connection and query abstraction
├── pms_client.lua        -- REST API client for PMS integration
├── tts_client.lua        -- TTS API client with caching
├── config_loader.lua     -- Tenant configuration loader
├── constants.lua        -- Shared constants and defaults
├── test/
│   ├── test_menu_navigator.lua
│   ├── test_dtmf_handler.lua
│   └── ...

```

4.2 Session Handling

The `ivr_main.lua` entry point follows this lifecycle:

```

-- ivr_main.lua (pseudocode)

local config = require("lib.config_loader")
local menu   = require("lib.menu_navigator")
local cdr    = require("lib.cdr_logger")

-- 1. Bootstrap
local domain_uuid = session:getVariable("domain_uuid")
local domain_name = session:getVariable("domain_name")
local caller_id   = session:getVariable("caller_id_number")
local dn timer     = session:getVariable("destination_number")

-- 2. Guard: verify session is active
if not session:ready() then return end

-- 3. Load tenant configuration from PostgreSQL
local tenant_config = config.load(domain_uuid)

-- 4. Initialize CDR context
local call_context = cdr.init({
    domain_uuid = domain_uuid,
    caller_id   = caller_id,
    dn timer    = dn timer,
    start_time  = os.time()
})

-- 5. Language selection (Feature 3.2)
local language = "en" -- MVP default; infrastructure ready for expansion

```

```

-- 6. Evaluate business hours (Features 2.1, 3.1)
local routing = require("lib.routing_engine")
local is_open, dept_routing = routing.evaluate_business_hours(domain_uuid, tenant_

-- 7. Play appropriate greeting
if is_open then
    prompt_player.play(session, dept_routing.business_hours_greeting, language)
else
    prompt_player.play(session, dept_routing.after_hours_greeting, language)
    routing.route_after_hours(session, dept_routing, call_context)
    cdr.finalize(call_context, "after_hours_routed")
    return
end

-- 8. Enter menu navigation loop
local result = menu.navigate(session, domain_uuid, "main_menu", language, call_con

-- 9. Finalize CDR
cdr.finalize(call_context, result.disposition)

```

4.3 DTMF Input Collection (Features 1.2, 1.5, 1.6, 1.7)

```

-- lib/dtmf_handler.lua (pseudocode)

local M = {}

function M.collect_single_digit(session, menu_config, prompt_file, valid_digits, l
    local max_retries_noinput = menu_config.max_noinput_retries or 3
    local max_retries_invalid = menu_config.max_invalid_retries or 3
    local timeout_ms = menu_config.initial_timeout_ms or 5000
    local noinput_count = 0
    local invalid_count = 0

    while session:ready() do
        local digit = session:playAndGetDigits(
            1, 1, -- min/max digits
            1, -- attempts per call to playAndGetDigits
            timeout_ms, -- timeout
            "", -- terminators (none for single digit)
            prompt_file, -- prompt to play
            "", -- no error file (handled below)
            "\\d|\\*|#", -- regex: any DTMF
            "ivr_digit", -- variable name
            3000 -- inter-digit timeout (not used for single)
        )
    end

```

```

    if not digit or digit == "" then
        -- No input received
        noinput_count = noinput_count + 1
        if noinput_count >= max_retries_noinput then
            return { result = "escalate", reason = "no_input_max_retries" }
        end
        -- Play progressive no-input prompt
        local noinput_prompt = M.get_noinput_prompt(noinput_count, language)
        session:streamFile(noinput_prompt)

    elseif not valid_digits[digit] then
        -- Invalid input
        invalid_count = invalid_count + 1
        if invalid_count >= max_retries_invalid then
            -- Final attempt: offer escape hatch
            local escape = M.offer_escape_hatch(session, language)
            if escape then return escape end
            return { result = "escalate", reason = "invalid_input_max_retries" }
        end
        local invalid_prompt = M.get_invalid_prompt(invalid_count, language)
        session:streamFile(invalid_prompt)

    else
        -- Valid input
        return { result = "valid", digit = digit }
    end
end

return { result = "hangup" }
end

function M.collect_multi_digit(session, min_digits, max_digits, timeout_ms, inter_
    local digits = session:playAndGetDigits(
        min_digits, max_digits,
        3, -- max attempts
        timeout_ms,
        "#", -- # terminates input
        prompt_file,
        "", -- error handled by caller
        "\\d+", -- digits only
        "ivr_multi_digit",
        inter_digit_ms
    )
    return digits
end

return M

```

4.4 Prompt Playback (Features 1.3, 1.4, 3.2)

```
-- lib/prompt_player.lua (pseudocode)

local tts = require("lib.tts_client")
local M = {}

function M.play(session, prompt_config, language)
    if not session:ready() then return end

    local file_path

    if prompt_config.source == "file" then
        -- Pre-recorded audio file (Feature 1.4)
        file_path = prompt_config.file_path
    elseif prompt_config.source == "tts" then
        -- TTS-generated audio (Feature 1.3)
        file_path = tts.get_or_generate(
            prompt_config.tts_text,
            language,
            prompt_config.voice or "female",
            prompt_config.domain_uuid
        )
    end

    if file_path and session:ready() then
        session:streamFile(file_path)
    else
        -- Fallback: use a generic system prompt
        session:streamFile("/usr/share/freeswitch/sounds/en/us/callie/ivr/ivr-gene
    end
end

return M
```

4.5 Retry Counter Logic

Retry counters are scoped per-menu-level. When the caller navigates to a sub-menu, new counters are initialized. When the caller presses * to return to a parent menu, that parent's counters are restored from the breadcrumb stack. This prevents counter accumulation across unrelated menu levels.

```
-- Breadcrumb stack entry
{
    menu_id          = "main_menu",
    level            = 1,
    noinput_counter   = 0,
    invalid_counter   = 0
}
```



```
}
```

4.6 Failover Routing

If the Lua script encounters an unrecoverable error (database connection failure, nil session, unhandled exception), it executes a failover route:

```
-- Global error handler wrapping ivr_main.lua
local ok, err = pcall(function()
    -- ... main IVR logic ...
end)

if not ok then
    freeswitch.consoleLog("ERR", "IVR error: " .. tostring(err))
    if session and session:ready() then
        session:streamFile("ivr/error_system_problem.wav")
        session:transfer("operator@" .. domain_name, "XML")
    end
end
end
```

4.7 Error Handling Strategy

Error Type	Handling
PostgreSQL connection failure	Retry once (500 ms backoff); if still failed, route to operator with system error prompt.
PMS API timeout (> 2 seconds)	Retry once with 1-second backoff; if failed, allow room dial without PMS validation (configurable: allow/deny/operator).
TTS API failure	Fall back to pre-recorded generic prompt or a cached version.
Session hangup mid-flow	All Lua functions check session:ready() before every I/O operation; graceful exit and CDR write on hangup.
Invalid database data (nil menu, missing config)	Log error; route to operator.

4.8 Logging Strategy

Lua scripts log to the FreeSWITCH console via `freeswitch.consoleLog()` with structured fields:

```
freeswitch.consoleLog("INFO",
    string.format("[IVR] domain=%s call_id=%s menu=%s action=%s digit=%s",
        domain_name, call_id, current_menu, action, digit))
```

Log levels used: - **ERR** — unrecoverable errors (DB failure, script crash) - **WARNING** — degraded operation (PMS timeout, TTS fallback) - **INFO** — call flow events (menu entry, DTMF received, transfer executed) - **DEBUG** — detailed tracing (DB query results, timing, config values) — disabled in production

4.9 Performance Considerations

1. **Database connection pooling:** Use `freeswitch.Dbh("pgsql://...")` which leverages FreeSWITCH's internal ODBC/native connection pool. Connections are reused across Lua invocations.
2. **Configuration caching:** Tenant config, menu trees, and business hours are loaded once at call start and held in local Lua variables for the call duration. No per-DTMF database round-trips.
3. **TTS caching:** Generated audio files are cached on disk with a hash-based filename. Cache lookups avoid redundant API calls (target > 80% cache hit rate).
4. **Minimal memory allocation:** Lua tables are pre-sized where possible. Large result sets are streamed, not buffered.
5. **Short-lived scripts:** Each call runs an isolated Lua instance. No global state leaks between calls.

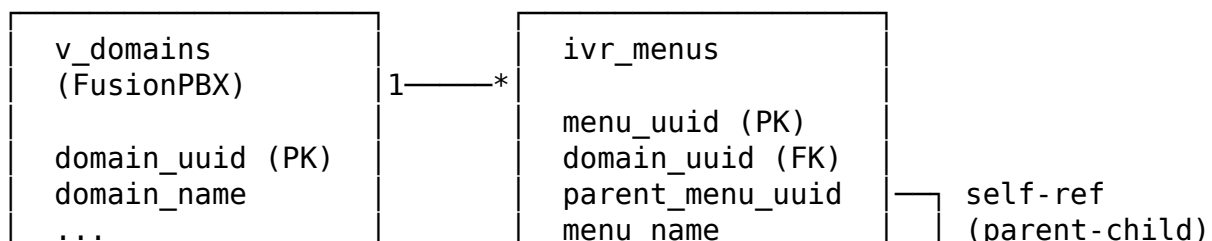
5. Database Design (FusionPBX PostgreSQL)

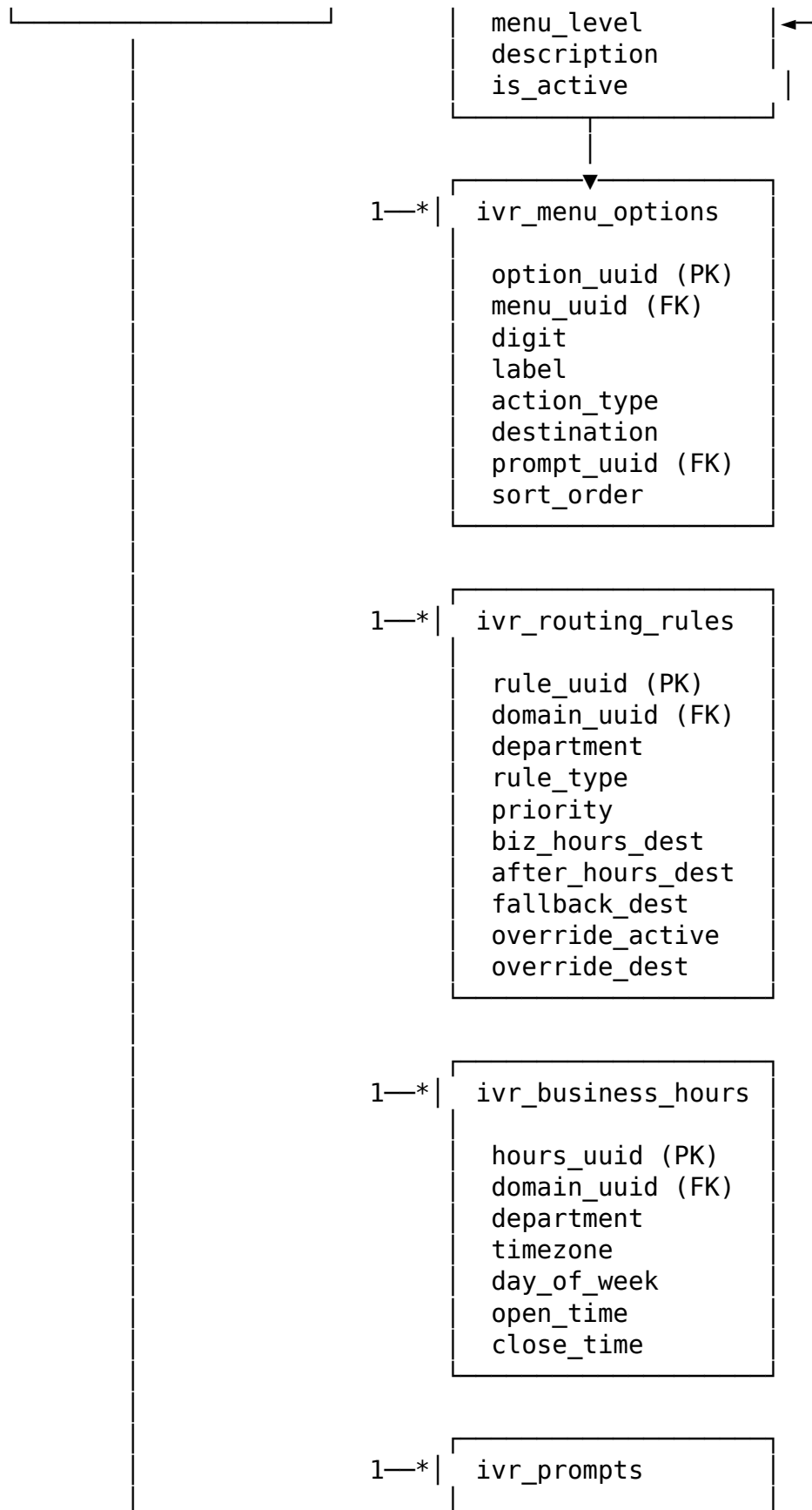
5.1 Schema Strategy

The IVR system extends the FusionPBX default PostgreSQL schema by adding custom tables in the same database. All custom tables:

- Are prefixed with `ivr_` for namespace clarity.
- Include `domain_uuid` as a mandatory foreign key for multi-tenant isolation.
- Follow FusionPBX conventions: UUID primary keys, `insert_date/update_date` timestamps.
- Never modify existing FusionPBX tables.

5.2 Entity Relationship Description





```

prompt_uuid (PK)
domain_uuid (FK)
prompt_name
prompt_type
language
source (file/tts)
file_path
tts_text
tts_voice
version
is_active

```

1—*

ivr_cdr

```

cdr_uuid (PK)
domain_uuid (FK)
call_uuid
caller_id
dnis
start_epoch
end_epoch
duration_sec
queue_time_sec
language
navigation_path
dtmf_sequence
final_destination
disposition
agent_id

```

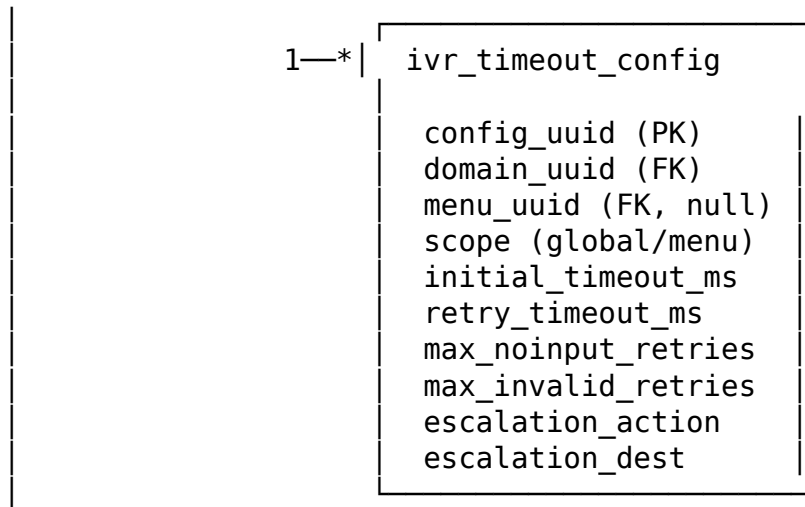
1—*

ivr_metrics_hourly

```

metric_uuid (PK)
domain_uuid (FK)
metric_hour
total_calls
answered_calls
abandoned_calls
voicemail_calls
avg_wait_seconds
avg_duration_seconds
department

```



5.3 Table Definitions

5.3.1 ivr_menus

Column	Type	Constraints	Description
menu_uuid	UUID	PK, DEFAULT gen_random_uuid()	Unique menu identifier
domain_uuid	UUID	FK → v_domains, NOT NULL	Tenant isolation
parent_menu_uuid	UUID	FK → ivr_menus, NULLABLE	Parent menu (NULL for root)
menu_name	VARCHAR(128)	NOT NULL	Human-readable name (e.g., “main_menu”)
menu_level	SMALLINT	NOT NULL, CHECK (1..5)	Nesting depth (MVP max 5)
description	TEXT	NULLABLE	Admin description
greeting_prompt_uuid	UUID	FK → ivr_prompts, NULLABLE	Prompt played on menu entry
is_active	BOOLEAN	NOT NULL DEFAULT true	Soft-enable/disable
insert_date	TIMESTAMPTZ	NOT NULL DEFAULT now()	
update_date	TIMESTAMPTZ		

5.3.2 ivr_menu_options

Column	Type	Constraints	Description
option_uuid	UUID	PK	Unique option identifier
menu_uuid	UUID	FK → ivr_menus, NOT NULL	Parent menu
domain_uuid	UUID	FK → v_domains, NOT NULL	Tenant isolation

Column	Type	Constraints	Description
digit	VARCHAR(2)	NOT NULL	DTMF key: 0-9, *, #
label	VARCHAR(128)	NOT NULL	Display label (e.g., “Front Desk”)
action_type	VARCHAR(32)	NOT NULL	One of: transfer, menu, room_dial, back, repeat, voicemail
destination	VARCHAR(256)	NULLABLE	Target: queue name, extension, sub-menu UUID, etc.
prompt_uuid	UUID	FK → ivr_prompts, NULLABLE	Option-specific prompt
sort_order	SMALLINT	NOT NULL DEFAULT 0	Display ordering
is_active	BOOLEAN	NOT NULL DEFAULT true	
insert_date	TIMESTAMP TZ	NOT NULL DEFAULT now()	

Unique constraint: (menu_uuid, digit) — no duplicate digits per menu.

5.3.3 ivr_routing_rules

Column	Type	Constraints	Description
rule_uuid	UUID	PK	
domain_uuid	UUID	FK → v_domains, NOT NULL	
department	VARCHAR(64)	NOT NULL	Department identifier (e.g., “front_desk”)
rule_type	VARCHAR(32)	NOT NULL	time_based, direct, overflow
priority	SMALLINT	NOT NULL DEFAULT 100	Lower number = higher priority
biz_hours_destination	VARCHAR(256)		Queue/ring group for business hours
after_hours_destination	VARCHAR(256)		Destination for after hours
fallback_destination	VARCHAR(256)		Final fallback (voicemail, operator)
queue_timeout_seconds	INTEGER	DEFAULT 45	Seconds before queue overflow
override_active	BOOLEAN	DEFAULT false	Manual override flag
override_destination	VARCHAR(256)		Override target
is_active	BOOLEAN	DEFAULT true	
insert_date	TIMESTAMP TZ	DEFAULT now()	
update_date	TIMESTAMP TZ		

5.3.4 ivr_business_hours

Column	Type	Constraints	Description
hours_uuid	UUID	PK	
domain_uuid	UUID	FK → v_domains, NOT NULL	
department	VARCHAR(64)	NOT NULL	
timezone	VARCHAR(64)	NOT NULL	IANA timezone (e.g., "America/New_York")
day_of_week	SMALLINT	NOT NULL, CHECK (0..6)	0=Sunday, 6=Saturday
open_time	TIME	NOT NULL	
close_time	TIME	NOT NULL	Can be past midnight (e.g., 01:00 for Friday close)
is_active	BOOLEAN	DEFAULT true	
insert_date	TIMESTAMP	DEFAULT now()	

Unique constraint: (domain_uuid, department, day_of_week).

Validation check: close_time > open_time OR close_time < open_time (cross-midnight is allowed: open 07:00, close 01:00 means open until 1 AM next day).

5.3.5 ivr_prompts

Column	Type	Constraints	Description
prompt_uuid	UUID	PK	
domain_uuid	UUID	FK → v_domains, NOT NULL	
prompt_name	VARCHAR(128)	NOT NULL	Identifier (e.g., "main_menu_greeting")
prompt_category	VARCHAR(64)		greeting, menu, error, system
language	VARCHAR(8)	NOT NULL DEFAULT 'en'	ISO 639-1 code
source	VARCHAR(16)	NOT NULL	file or tts
file_path	VARCHAR(512)		Filesystem path for uploaded audio
tts_text	TEXT		Text content for TTS generation
tts_voice	VARCHAR(64)		Voice identifier (e.g., "en-US-Wavenet-F")
tts_engine	VARCHAR(32)		google or polly
format	VARCHAR(16)		wav, mp3
sample_rate	INTEGER		8000, 16000, 44100
file_size_bytes	INTEGER		
version	INTEGER	NOT NULL DEFAULT 1	Incremented on update

Column	Type	Constraints	Description
is_active	BOOLEAN	DEFAULT true	
insert_date	TIMESTAMPTZ	DEFAULT now()	
update_date	TIMESTAMPTZ		

5.3.6 ivr_cdr

Column	Type	Constraints	Description
cdr_uuid	UUID	PK	
domain_uuid	UUID	FK → v_domains, NOT NULL	
call_uuid	VARCHAR(64)	NOT NULL, UNIQUE	FreeSWITCH call UUID
caller_id	VARCHAR(32)		ANI
dnis	VARCHAR(32)		Dialed number
start_epoch	BIGINT	NOT NULL	Unix epoch start time
answer_epoch	BIGINT		Unix epoch answer time
end_epoch	BIGINT		Unix epoch end time
duration_seconds	INTEGER		Total call duration
queue_time_seconds	INTEGER		Time spent in queue
language	VARCHAR(8)	DEFAULT 'en'	Selected language
navigation_path	JSONB		Array of menu_ids traversed
dtmf_sequence	JSONB		Array of DTMF inputs
final_destination	VARCHAR(256)		Where the call was delivered
disposition	VARCHAR(32)	NOT NULL	answered, abandoned, voicemail, error, after_hours_routed
agent_id	VARCHAR(128)		Agent who answered (if applicable)
department	VARCHAR(64)		Department routed to
pms_room_queried	VARCHAR(16)		Room number queried (if applicable)
insert_date	TIMESTAMPTZ	DEFAULT now()	

5.3.7 ivr_metrics_hourly

Column	Type	Constraints	Description
metric_uuid	UUID	PK	
domain_uuid	UUID	FK → v_domains, NOT NULL	
metric_hour	TIMESTAMPTZ	NOT NULL	Truncated to hour
department	VARCHAR(64)		NULL for aggregate totals
total_calls	INTEGER	DEFAULT 0	

Column	Type	Constraints	Description
answered_calls	INTEGER	DEFAULT 0	
abandoned_calls	INTEGER	DEFAULT 0	
voicemail_calls	INTEGER	DEFAULT 0	
error_calls	INTEGER	DEFAULT 0	
avg_wait_seconds	NUMERIC(8,2)		
avg_duration_seconds	NUMERIC(8,2)		
max_wait_seconds	INTEGER		
service_level_percent	NUMERIC(5,2)		% answered within SLA threshold
insert_date	TIMESTAMPTZ	DEFAULT now()	
update_date	TIMESTAMPTZ		

Unique constraint: (domain_uuid, metric_hour, department).

5.3.8 ivr_timeout_config

Column	Type	Constraints	Description
config_uuid	UUID	PK	
domain_uuid	UUID	FK → v_domains, NOT NULL	
menu_uuid	UUID	FK → ivr_menus, NULLABLE	NULL = global config
scope	VARCHAR(16)	NOT NULL	global or menu
initial_timeout_ms	INTEGER	CHECK (3000..10000)	Default 5000
retry_timeout_ms	INTEGER	CHECK (3000..10000)	Default 5000
max_noinput_retries	SMALLINT	CHECK (1..5)	Default 3
max_invalid_retries	SMALLINT	CHECK (1..5)	Default 3
escalation_action	VARCHAR(32)		operator, voicemail, disconnect
escalation_destination	VARCHAR(256)		Target for escalation
insert_date	TIMESTAMPTZ	DEFAULT now()	
update_date	TIMESTAMPTZ		

5.4 Indexing Strategy

```
-- Multi-tenant isolation (critical for every query)
CREATE INDEX idx_ivr_menus_domain ON ivr_menus (domain_uuid);
CREATE INDEX idx_ivr_menu_options_menu ON ivr_menu_options (menu_uuid);
CREATE INDEX idx_ivr_menu_options_domain ON ivr_menu_options (domain_uuid);

-- Routing rule lookups (per call)
CREATE INDEX idx_ivr_routing_rules_domain_dept ON ivr_routing_rules (domain_uuid, department_uuid);
CREATE INDEX idx_ivr_business_hours_domain_dept ON ivr_business_hours (domain_uuid, department_uuid);
```

```

-- CDR queries (reporting)
CREATE INDEX idx_ivr_cdr_domain_start ON ivr_cdr (domain_uuid, start_epoch DESC);
CREATE INDEX idx_ivr_cdr_domain_disposition ON ivr_cdr (domain_uuid, disposition);
CREATE INDEX idx_ivr_cdr_call_uuid ON ivr_cdr (call_uuid);
CREATE INDEX idx_ivr_cdr_insert_date ON ivr_cdr (insert_date DESC);

-- Metrics aggregation queries
CREATE INDEX idx_ivr_metrics_domain_hour ON ivr_metrics_hourly (domain_uuid, metric_hour);
CREATE INDEX idx_ivr_metrics_dept_hour ON ivr_metrics_hourly (domain_uuid, department_hour);

-- Prompt lookups
CREATE INDEX idx_ivr_prompts_domain_name ON ivr_prompts (domain_uuid, prompt_name, prompt_text);

-- Timeout config resolution (global then menu-specific)
CREATE INDEX idx_ivr_timeout_domain_scope ON ivr_timeout_config (domain_uuid, scope);

```

5.5 Multi-Tenant Isolation via domain_uuid

Every query executed by the Lua scripts includes domain_uuid in the WHERE clause. The domain_uuid is extracted from the FreeSWITCH channel variable at call setup and propagated through all database helper functions:

```

-- lib/db_helper.lua
function M.query(dbh, sql, domain_uuid, params)
    -- domain_uuid is ALWAYS the first bind parameter
    -- Lua scripts never execute cross-domain queries
    local full_sql = sql .. " AND domain_uuid = ?"
    -- ...
end

```

Row-Level Security (RLS) is **recommended** as a defense-in-depth measure:

```

ALTER TABLE ivr_menus ENABLE ROW LEVEL SECURITY;
CREATE POLICY ivr_menus_tenant_isolation ON ivr_menus
    USING (domain_uuid = current_setting('app.current_domain_uuid')::uuid);

```

5.6 Migration Strategy

Schema migrations are managed via numbered SQL scripts stored in the Git repository:

```

migrations/
├── 001_create_ivr_menus.sql
├── 002_create_ivr_menu_options.sql
├── 003_create_ivr_routing_rules.sql
├── 004_create_ivr_business_hours.sql
├── 005_create_ivr_prompts.sql
├── 006_create_ivr_cdr.sql
└── 007_create_ivr_metrics_hourly.sql

```

```

├── 008_create_ivr_timeout_config.sql
├── 009_create_indexes.sql
└── ...

```

A schema_version table tracks applied migrations:

```

CREATE TABLE ivr_schema_version (
  version INTEGER PRIMARY KEY,
  description VARCHAR(256),
  applied_at TIMESTAMPTZ DEFAULT now()
);

```

Migrations are applied via a shell script during deployment, running each unapplied migration in a transaction with rollback on failure. FusionPBX's existing schema is never modified.

6. Prompt & Media Management

6.1 Audio Storage Location

Audio files follow the FusionPBX filesystem convention:

```

/var/lib/freeswitch/recordings/{domain_name}/
├── ivr/
│   ├── greetings/
│   │   ├── main_menu_greeting_v1.wav
│   │   ├── after_hours_greeting_v1.wav
│   │   └── ...
│   ├── menus/
│   │   ├── main_menu_options_v1.wav
│   │   ├── room_dialing_prompt_v1.wav
│   │   └── ...
│   ├── errors/
│   │   ├── invalid_input_1.wav
│   │   ├── invalid_input_2.wav
│   │   ├── invalid_input_3.wav
│   │   ├── no_input_1.wav
│   │   ├── no_input_2.wav
│   │   ├── no_input_3.wav
│   │   └── system_error.wav
│   └── system/
│       ├── connecting.wav
│       ├── room_not_available.wav
│       ├── room_dnd_active.wav
│       ├── voicemail_greeting.wav
│       └── goodbye.wav
└── voicemail/
    └── {extension}/

```

└─ msg_{timestamp}.wav

6.2 TTS Integration (Feature 1.3)

```
-- lib/tts_client.lua (pseudocode)

local M = {}

local TTS_CACHE_DIR = "/var/lib/freeswitch/tts_cache/"
local TTS_API_TIMEOUT = 2000 -- 2 seconds

function M.get_or_generate(text, language, voice, domain_uuid)
    -- Generate cache key from content hash
    local cache_key = M.hash(text .. language .. voice)
    local cache_path = TTS_CACHE_DIR .. domain_uuid .. "/" .. cache_key .. ".wav"

    -- Check cache first
    if M.file_exists(cache_path) then
        return cache_path -- Cache hit
    end

    -- Cache miss: call TTS API
    local audio_data = M.call_tts_api(text, language, voice)
    if audio_data then
        M.write_file(cache_path, audio_data)
        return cache_path
    end

    -- TTS failure: return nil (caller should use fallback)
    return nil
end

function M.call_tts_api(text, language, voice)
    -- HTTP POST to Google Cloud TTS or AWS Polly
    -- Returns raw audio bytes (LINEAR16 PCM, 8kHz for telephony)
    -- Timeout: 2 seconds
    -- ...
end

return M
```

6.2.1 TTS Client Architecture

6.2.2 TTS Provider Configuration

Parameter	Value
Primary provider	Google Cloud TTS (recommended)
Fallback provider	AWS Polly (if Google unavailable)
Output format	LINEAR16 PCM WAV, 8kHz mono
Voices (MVP)	Female: en-US-Wavenet-F; Male: en-US-Wavenet-D
API timeout	2 seconds
Max text length	5000 characters per request

6.3 Cache Strategy

- **Cache key:** SHA-256 hash of (text + language + voice).
- **Cache location:** /var/lib/freeswitch/tts_cache/{domain_uuid}/.
- **Cache invalidation:** When a prompt's TTS text is updated in the database (version incremented), the old cache file is deleted and regenerated on next access.
- **Cache warm-up:** On deployment or prompt update, a background job pre-generates all TTS prompts to avoid first-call latency.
- **Target cache hit rate:** > 80%.
- **Cache eviction:** Files not accessed for 30 days are purged via a daily cron job.

6.4 Prompt Versioning

Each prompt in `ivr_prompts` has a version integer that increments on every update. The file naming convention encodes the version:

`{prompt_name}_v{version}.wav`

Examples: - `main_menu_greeting_v1.wav` - `main_menu_greeting_v2.wav`

When a new version is published: 1. The new file is written alongside the old one. 2. The `ivr_prompts` record is updated with the new `file_path` and incremented version. 3. New calls immediately use the new version (no restart required). 4. Old versions are retained for 7 days for rollback, then purged.

6.5 File Naming Convention

`{category}_{identifier}[_{language}]_v{version}.{format}`

Component	Convention	Example
category	greeting, menu, error, system	greeting
identifier	snake_case descriptive name	main_menu
language	ISO 639-1 code (omitted if en)	es
version	Integer, prefixed with v	v3
format	wav or mp3	wav

Example: greeting_main_menu_v2.wav, menu_room_dialing_es_v1.wav

6.6 Audio File Upload (Feature 1.4)

Upload validation checklist:

Check	Criteria	Action on Failure
Format	WAV (PCM) or MP3	Reject with error message
Sample rate	8 kHz, 16 kHz, or 44.1 kHz	Accept; transcode to 8 kHz for telephony
Channels	Mono or stereo	Accept; downmix to mono
File size	≤ 10 MB	Reject with error message
Duration	≤ 300 seconds (5 minutes)	Reject with error message
Audio quality	No clipping (peak < 0 dBFS), no pure silence	Warn admin; allow override

Uploaded files are stored in the domain's prompt directory and registered in `ivr_prompts`.

6.7 Multi-Language Infrastructure (Feature 3.2)

Although MVP is English-only, the architecture supports multi-language:

- Prompts are keyed by (`prompt_name`, `language`) in `ivr_prompts`.
- The Lua prompt player resolves prompts by `prompt_name` + `language` session variable.
- If a prompt is not found for the requested language, it falls back to `en`.
- Directory structure includes language subdirectories (ready for Phase 2):

```
ivr/greetings/en/main_menu_greeting_v1.wav
ivr/greetings/es/main_menu_greeting_v1.wav    # Phase 2
```

7. Routing Engine Design (Within FusionPBX)

7.1 Routing Decision Hierarchy

The routing engine follows a strict priority order, evaluated in the Lua script:

Priority 1: Manual override check

└─ If `override_active` = true → route to `override_destination` (skip all else)

Priority 2: Business hours evaluation (Features 2.1, 3.1)

└─ Determine if current time is within department business hours

└─ Open → use `biz_hours_destination`

└─ Closed → use `after_hours_destination`

Priority 3: Target resolution

- └─ Resolve destination string to FusionPBX entity:
 - └─ "front_desk_queue" → mod_fifo queue
 - └─ "ext:1001" → SIP extension (ring group or direct)
 - └─ "room:1234" → Room phone extension (via PMS validation)
 - └─ "vm:front_desk" → Department voicemail box
 - └─ "operator" → Operator extension (0 key escape hatch)

Priority 4: Fallback chain

- └─ If primary destination fails (timeout, no answer, error):
 - └─ Try fallback_destination from routing rule
 - └─ If fallback fails → route to domain operator extension

7.2 Domain-Based Routing

All routing is scoped to the FusionPBX domain (tenant). The Lua routing engine:

1. Loads routing rules filtered by domain_uuid.
2. Resolves destination names to domain-local FusionPBX entities (extensions, ring groups, queues).
3. Transfers calls within the domain's XML dialplan context:


```
session:transfer(destination, "XML", domain_name)
```
4. Cross-domain routing is explicitly prohibited in MVP.

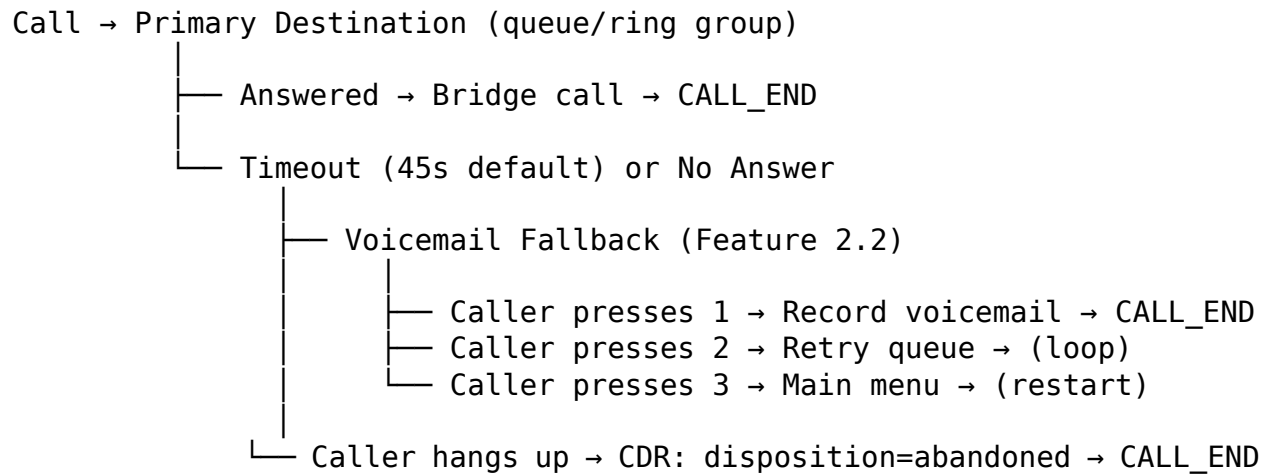
7.3 Ring Group vs. FIFO Queue Selection

Scenario	Mechanism	Rationale
Small team (2-5 agents), all should ring simultaneously	FusionPBX Ring Group	Simultaneous or sequential ring with quick answer time. No queue wait experience.
Department with variable staffing, callers may need to wait	mod_fifo	FIFO queue with hold music, configurable timeout, and voicemail overflow.
Night manager (single person)	Direct extension transfer	No queue overhead; direct SIP INVITE to the extension or mobile.

The `ivr_routing_rules.biz_hours_destination` field encodes the mechanism:

- `ringgroup:{ring_group_uuid}` — transfer to FusionPBX ring group.
- `fifo:{fifo_name}` — transfer to mod_fifo queue.
- `ext:{extension}` — direct transfer to SIP extension.
- `vm:{voicemail_id}` — direct to voicemail.

7.4 Fallback Routing Strategy



7.5 Queue Overflow Logic

When a mod_fifo queue times out:

1. FreeSWITCH executes the FIFO timeout action, which transfers back to a Lua handler script (ivr/queue_overflow.lua).
2. The overflow handler plays the voicemail option menu (Feature 2.2).
3. If the caller chooses to wait, they are re-inserted into the FIFO.
4. If max re-entries exceeded (configurable, default 2), force voicemail or operator.

7.6 Voicemail Fallback (Feature 2.2)

```

-- lib/voicemail_handler.lua (pseudocode)

function M.offer_voicemail(session, department, domain_name, call_context)
    local prompt = "ivr/system/voicemail_option.wav"
    -- "All agents are busy. Press 1 for voicemail, 2 to try again, 3 for main menu"

    local digit = dtmf_handler.collect_single_digit(session, default_config, prompt,
        { ["1"] = true, ["2"] = true, ["3"] = true }, "en")

    if digit.result == "valid" then
        if digit.digit == "1" then
            -- Transfer to FusionPBX voicemail for this department
            session:transfer("*99" .. department_extension, "XML", domain_name)
            call_context.disposition = "voicemail"
        elseif digit.digit == "2" then
            return "retry_queue"
        elseif digit.digit == "3" then
            return "main_menu"
        end
    else

```



```

    -- Escalate to operator
    session:transfer("operator@" .. domain_name, "XML")
end
end

```

7.7 Implementation Across FusionPBX Components

Routing Function	Implemented In	Details
DID → IVR launch	Dialplan XML	Public context matches DID, sets domain vars, launches Lua.
Menu navigation	Lua (menu_navigator.lua)	Reads menu tree from PostgreSQL, handles DTMF, tracks breadcrumbs.
Business hour check	Lua (routing_engine.lua)	Queries <code>ivr_business_hours</code> , evaluates time, selects destination.
Department queue	mod_fifo	Lua transfers to FIFO queue. <code>mod_fifo</code> manages hold music and agent dispatch.
Ring group	FusionPBX ring group	Lua transfers to ring group extension. FusionPBX handles ring strategy.
Room routing	Lua (room_router.lua)	Queries PMS API, validates room, bridges to room extension via <code>session:execute("bridge")</code> .
Voicemail	mod_voicemail	Lua transfers to voicemail extension. FusionPBX records, stores, and emails.

Routing Function	Implemented In	Details
Operator escape	Dialplan XML	Operator extension defined per domain; Lua transfers to it.

8. Reporting & Metrics

8.1 CDR Extraction (Feature 5.1)

The system captures CDR data at two levels:

1. **FreeSWITCH XML CDR** (v_xml_cdr table) — standard SIP-level records (caller, callee, duration, codecs, hangup cause). Written automatically by FreeSWITCH.
2. **IVR Enriched CDR** (ivr_cdr table) — application-level records with navigation path, DTMF sequence, disposition, queue time, and department. Written by the Lua cdr_logger at call completion.

The enriched CDR joins with the XML CDR via call_uuid for complete call forensics.

8.2 Data Aggregation Approach

Hourly metrics are pre-aggregated into ivr_metrics_hourly by a PostgreSQL scheduled job (using pg_cron or an external cron):

```
-- Hourly aggregation job (runs at :05 past every hour)
INSERT INTO ivr_metrics_hourly (
    metric_uuid, domain_uuid, metric_hour, department,
    total_calls, answered_calls, abandoned_calls, voicemail_calls, error_calls,
    avg_wait_seconds, avg_duration_seconds, max_wait_seconds, service_level_pct
)
SELECT
    gen_random_uuid(),
    domain_uuid,
    date_trunc('hour', to_timestamp(start_epoch)) AS metric_hour,
    department,
    COUNT(*) AS total_calls,
    COUNT(*) FILTER (WHERE disposition = 'answered') AS answered_calls,
    COUNT(*) FILTER (WHERE disposition = 'abandoned') AS abandoned_calls,
    COUNT(*) FILTER (WHERE disposition = 'voicemail') AS voicemail_calls,
    COUNT(*) FILTER (WHERE disposition = 'error') AS error_calls,
    AVG(queue_time_seconds) AS avg_wait_seconds,
    AVG(duration_seconds) AS avg_duration_seconds,
    MAX(queue_time_seconds) AS max_wait_seconds,
    (COUNT(*) FILTER (WHERE disposition = 'answered' AND queue_time_seconds <= 30)
    / NULLIF(COUNT(*), 0) * 100) AS service_level_pct
```

```

FROM ivr_cdr
WHERE start_epoch >= EXTRACT(EPOCH FROM date_trunc('hour', now() - interval '1 hou
    AND start_epoch < EXTRACT(EPOCH FROM date_trunc('hour', now()))
GROUP BY domain_uuid, metric_hour, department
ON CONFLICT (domain_uuid, metric_hour, department)
DO UPDATE SET
    total_calls = EXCLUDED.total_calls,
    answered_calls = EXCLUDED.answered_calls,
    abandoned_calls = EXCLUDED.abandoned_calls,
    voicemail_calls = EXCLUDED.voicemail_calls,
    error_calls = EXCLUDED.error_calls,
    avg_wait_seconds = EXCLUDED.avg_wait_seconds,
    avg_duration_seconds = EXCLUDED.avg_duration_seconds,
    max_wait_seconds = EXCLUDED.max_wait_seconds,
    service_level_pct = EXCLUDED.service_level_pct,
    update_date = now();

```

8.3 Reporting Queries

```

SELECT COUNT(*) AS total_calls
FROM ivr_cdr
WHERE domain_uuid = :domain_uuid
    AND start_epoch >= EXTRACT(EPOCH FROM date_trunc('day', now() AT TIME ZONE :tz))

```

Total Calls (Today, Per Domain)

```

SELECT
    COUNT(*) FILTER (WHERE disposition = 'abandoned')::NUMERIC
    / NULLIF(COUNT(*), 0) * 100 AS abandonment_rate_pct
FROM ivr_cdr
WHERE domain_uuid = :domain_uuid
    AND start_epoch >= EXTRACT(EPOCH FROM date_trunc('day', now() AT TIME ZONE :tz))

```

Abandonment Rate (Today)

```

SELECT AVG(queue_time_seconds) AS avg_wait_seconds
FROM ivr_cdr
WHERE domain_uuid = :domain_uuid
    AND queue_time_seconds IS NOT NULL
    AND start_epoch >= EXTRACT(EPOCH FROM date_trunc('day', now() AT TIME ZONE :tz))

```

Average Wait Time (Today)

```

SELECT
  (COUNT(*) FILTER (WHERE disposition = 'answered' AND queue_time_seconds <= 30)
   / NULLIF(COUNT(*) FILTER (WHERE disposition IN ('answered', 'abandoned')), 0)
  AS service_level_pct
FROM ivr_cdr
WHERE domain_uuid = :domain_uuid
  AND start_epoch >= EXTRACT(EPOCH FROM date_trunc('day', now()) AT TIME ZONE :tz))

```

Service Level % (Answered within 30 seconds)

```

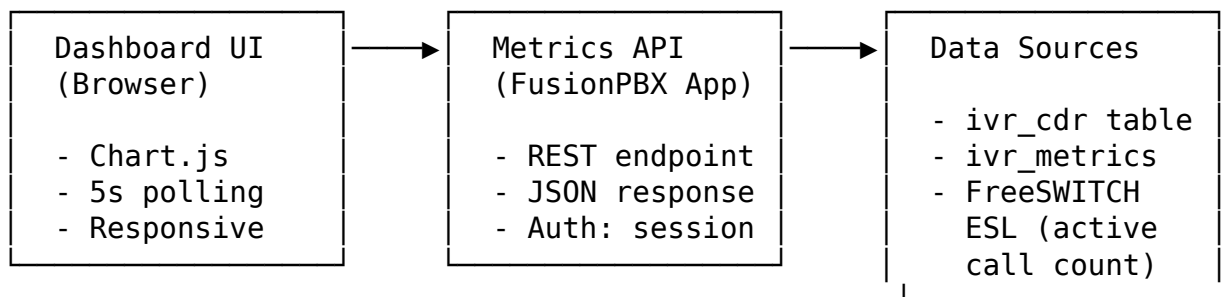
SELECT
  date_trunc('hour', to_timestamp(start_epoch)) AS hour,
  COUNT(*) AS call_count
FROM ivr_cdr
WHERE domain_uuid = :domain_uuid
  AND start_epoch >= EXTRACT(EPOCH FROM date_trunc('day', now()) AT TIME ZONE :tz))
GROUP BY hour
ORDER BY call_count DESC
LIMIT 1;

```

Peak Hour (Today)

8.4 Real-Time Metrics Dashboard (Feature 5.2)

8.4.1 Architecture



8.4.2 Metrics Collected

Metric	Source	Refresh
Active calls (live)	FreeSWITCH ESL show calls count	5 seconds
Calls in IVR (live)	ESL: calls with application=lua	5 seconds
Calls in queue (live)	ESL: fifo list count	5 seconds
Total calls today	ivr_cdr COUNT	5 seconds
Abandonment rate today	ivr_cdr filtered COUNT	5 seconds
Average wait time today	ivr_cdr AVG(queue_time)	5 seconds

Metric	Source	Refresh
Department breakdown	ivr_cdr GROUP BY department	30 seconds
Hourly call volume chart	ivr_metrics_hourly	60 seconds
Service level %	ivr_cdr filtered calculation	30 seconds

8.4.3 Daily Summary Generation A cron job at 00:15 (hotel local time) generates a daily summary by aggregating the hourly metrics:

```
0 15 0 * * * /usr/share/freeswitch/scripts/ivr/jobs/daily_summary.lua
```

The summary is stored in a `ivr_daily_summary` view or materialized query and can be exported to CSV via the admin UI.

8.4.4 Alert Thresholds

Condition	Threshold	Action
Abandonment rate spike	> 15% in last 30 min	Dashboard warning indicator
Queue wait time	> 120 seconds average in last 15 min	Dashboard critical indicator
Call volume spike	> 200% of same-hour average	Dashboard info indicator
Zero active agents	0 agents logged into FIFO	Dashboard critical indicator

Alerts are displayed on the dashboard UI. Email alerting is deferred to Phase 2.

9. Security & Compliance

9.1 SIP Security (Feature 6.1)

9.1.1 Signaling Encryption

Parameter	Value
Protocol	SIP over TLS (SIPS)
Port	5061 (external profile), 5061 (internal profile)
TLS version	TLS 1.2 minimum; TLS 1.3 preferred
Cipher suites	TLS_AES_256_GCM_SHA384, TLS_AES_128_GCM_SHA256, TLS_CHACHA20_POLY1305_SHA256

Parameter	Value
Certificate	Wildcard or SAN cert from trusted CA (Let's Encrypt for MVP)
Auto-renewal	Certbot with cron renewal (60-day cycle)
Mutual TLS (mTLS)	Supported; enabled for high-security carrier trunks
Legacy TLS (1.0/1.1)	Disabled
Unencrypted SIP (UDP/TCP 5060)	Disabled in production

FusionPBX mod_sofia profile configuration:

```
<param name="tls" value="true"/>
<param name="tls-bind-params" value="transport=tls"/>
<param name="tls-sip-port" value="5061"/>
<param name="tls-cert-dir" value="/etc/freeswitch/tls"/>
<param name="tls-version" value="tlsv1.2,tlsv1.3"/>
```

9.1.2 Media Encryption

Parameter	Value
Protocol	SRTP (RFC 3711)
Key exchange	SDS (SDP Security Descriptions) for compatibility; DTLS-SRTP where supported
Cipher	AES_CM_128_HMAC_SHA1_80
Policy	Mandatory — calls failing SRTP negotiation are rejected
Fallback to RTP	Disabled in production

Sofia profile setting:

```
<param name="rtp-secure-media" value="mandatory"/>
```

9.2 PostgreSQL Access Control

Measure	Implementation
Network access	PostgreSQL listens on private subnet only; no public IP binding
Authentication	scram-sha-256 (pg_hba.conf); no trust or md5
Connection encryption	sslmode=require for all clients

Measure	Implementation
Application user	Dedicated <code>ivr_app</code> role with minimal privileges: SELECT/INSERT/UPDATE on <code>ivr_*</code> tables; no DDL, no superuser
Admin user	<code>fusionpbx</code> role for schema management; accessed only via admin tooling
Connection pooling	PgBouncer in front of PostgreSQL; connection limit per user

9.3 FusionPBX RBAC Roles

Role	Permissions
Superadmin	Full system access; manage all domains, all IVR configs, schema migrations
Domain Admin (per hotel)	Manage IVR menus, prompts, business hours, routing rules, and view CDR for their domain only
Supervisor	View real-time dashboard and CDR reports for their domain; no configuration changes
Agent	No IVR admin access; receive calls via queue/ring group

FusionPBX enforces role-based access via its built-in group/permission system. IVR admin pages are registered under a new permission group (`ivr_admin`).

9.4 API Authentication

Integration	Auth Method	Details
PMS REST API	API key + HTTPS	API key stored in <code>ivr_integration_config</code> (encrypted column or environment variable). All PMS calls over HTTPS with certificate validation.
TTS Cloud API	Service account / API key + HTTPS	Google Cloud service account JSON or AWS IAM credentials. Stored as environment variables on the FusionPBX node, not in database.

Integration	Auth Method	Details
Metrics Dashboard API	FusionPBX session cookie	Dashboard is a FusionPBX app page; authentication via FusionPBX login session.
FreeSWITCH ESL	Localhost-only + password	ESL bound to 127.0.0.1:8021 with a strong password in <code>event_socket.conf.xml</code> .

9.5 Audit Logging

All administrative actions are logged:

Event	Logged Data
IVR menu create/update/delete	Admin user, domain, timestamp, before/after values
Prompt upload/update	Admin user, domain, filename, version
Business hours change	Admin user, domain, department, old/new schedule
Routing rule change	Admin user, domain, before/after config
Manual override toggle	Admin user, domain, department, override status

Audit logs are stored in a dedicated `ivr_audit_log` table with 1-year retention.

9.6 Tenant Isolation

- **Database:** All queries filtered by `domain_uuid`. Row-Level Security policies enforced.
- **Filesystem:** Audio files segregated by domain directory. File permissions restrict cross-domain access.
- **SIP:** Each domain has distinct DID numbers and SIP profiles. Calls cannot cross domains.
- **Admin UI:** FusionPBX enforces domain-scoped views. Domain admins cannot see other domains' data.

9.7 Firewall and Fail2Ban

Firewall Rules (iptables / nftables)

Port	Protocol	Source	Purpose
5061	TCP (TLS)	Carrier IPs (allowlist)	SIP signaling
10000-20000	UDP	Carrier IPs	RTP/SRTP media
443	TCP	Admin network	FusionPBX web UI

Port	Protocol	Source	Purpose
8021	TCP	127.0.0.1 only	FreeSWITCH ESL
5432	TCP	App nodes only (private subnet)	PostgreSQL
22	TCP	Admin jump host only	SSH

All other ports: **DENY**.

```
[freeswitch-dos]
enabled = true
filter = freeswitch
logpath = /var/log/freeswitch/freeswitch.log
maxretry = 5
findtime = 60
bantime = 3600
action = iptables-allports[name=freeswitch, protocol=all]

[fusionpbx-auth]
enabled = true
filter = fusionpbx
logpath = /var/log/fusionpbx/login_failed.log
maxretry = 5
findtime = 300
bantime = 7200
```

Fail2Ban Configuration SIP registration brute-force protection: FreeSWITCH's built-in log-auth-failures parameter combined with Fail2Ban regex matching on AUTH_FAILURE log entries.

10. Performance & Scalability

10.1 Expected Concurrent Call Handling

Deployment Size	Concurrent Calls	Properties Served
Single node (MVP start)	200-500	1-10 hotels
Two-node cluster	500-1000	10-25 hotels
Multi-node (Phase 2)	1000+	25+ hotels

Sizing is based on FreeSWITCH benchmarks: ~500 concurrent bridged calls per 8-vCPU node with Lua IVR processing.

10.2 FreeSWITCH Performance Tuning

```
<!-- switch.conf.xml -->
<param name="max-sessions" value="2000"/>
<param name="sessions-per-second" value="100"/>
<param name="rtp-start-port" value="10000"/>
<param name="rtp-end-port" value="20000"/>

<!-- sofia profile (external) -->
<param name="inbound-codec-prefs" value="PCMU,PCMA,G729"/>
<param name="outbound-codec-prefs" value="PCMU,PCMA,G729"/>
<param name="rtp-timer-name" value="soft"/>
<param name="nonce-ttl" value="60"/>
```

OS-level tuning (sysctl):

```
fs.file-max = 1000000
net.core.rmem_max = 16777216
net.core.wmem_max = 16777216
net.core.somaxconn = 65535
net.ipv4.tcp_max_syn_backlog = 65535
net.ipv4.ip_local_port_range = 1024 65535
```

Ulimits for the freeswitch user:

```
freeswitch soft nofile 1000000
freeswitch hard nofile 1000000
freeswitch soft core unlimited
```

10.3 CPU/Memory Sizing

Component	CPU Guideline	Memory Guideline
FreeSWITCH + mod_lua	1 vCPU per ~60 concurrent calls with Lua processing	32 MB base + ~0.5 MB per concurrent call
PostgreSQL	1 vCPU per ~200 concurrent queries	2 GB shared_buffers for 8 GB RAM node
SIP LB (Kamailio)	1 vCPU per ~1000 concurrent SIP transactions	1 GB
TTS cache I/O	Minimal (disk-bound, SSD resolves)	N/A

10.4 PostgreSQL Optimization

```
-- postgresql.conf tuning for IVR workload

-- Memory
```

```

shared_buffers = '2GB'           -- 25% of total RAM
effective_cache_size = '6GB'     -- 75% of total RAM
work_mem = '64MB'               -- per-query sort/hash memory
maintenance_work_mem = '512MB'  -- for VACUUM, CREATE INDEX

-- Write-ahead log
wal_buffers = '64MB'
checkpoint_completion_target = 0.9
max_wal_size = '4GB'

-- Query planner
random_page_cost = 1.1          -- SSD storage
effective_io_concurrency = 200   -- SSD parallel I/O

-- Connections
max_connections = 200            -- via PgBouncer, actual client count higher

```

10.5 Connection Pooling

PgBouncer configuration:

```

[databases]
fusionpbx = host=127.0.0.1 port=5432 dbname=fusionpbx

[pgbouncer]
listen_port = 6432
listen_addr = 127.0.0.1
auth_type = scram-sha-256
pool_mode = transaction
max_client_conn = 1000
default_pool_size = 50
reserve_pool_size = 10
reserve_pool_timeout = 3
server_lifetime = 3600
server_idle_timeout = 600

```

FreeSWITCH Lua scripts connect via PgBouncer (port 6432) rather than directly to PostgreSQL. This limits the actual PostgreSQL connection count while supporting hundreds of concurrent Lua sessions.

10.6 Lua Script Performance Considerations

Concern	Mitigation
Database queries per call	Batch-load config at call start (1–3 queries total); no per-DTMF queries.

Concern	Mitigation
String concatenation in loops	Use <code>table.concat()</code> instead of <code>..</code> operator for building large strings.
TTS API latency	Cache on disk; pre-warm cache on deploy. First-call latency: ~500 ms (P95); cached: 0 ms.
PMS API latency	2-second timeout with 1 retry; circuit-breaker pattern for sustained failures.
Memory per call	Target < 2 MB Lua heap per active IVR session.
Garbage collection	Use <code>collectgarbage("step")</code> between menu levels to prevent GC pauses during prompt playback.

10.7 CDR Indexing Strategy

Indexes are designed for the two primary access patterns:

1. **Real-time dashboard queries** — filter by `domain_uuid` + time range + disposition. Covered by `idx_ivr_cdr_domain_start` and `idx_ivr_cdr_domain_disposition`.
2. **Admin search/export** — filter by `domain_uuid` + time range + optional `caller_id/agent_id`. The `start_epoch DESC` index supports reverse-chronological listing efficiently.

CDR table is partitioned by month (PostgreSQL declarative partitioning) to manage data volume:

```
CREATE TABLE ivr_cdr (
    -- ... columns ...
) PARTITION BY RANGE (start_epoch);

CREATE TABLE ivr_cdr_2026_02 PARTITION OF ivr_cdr
    FOR VALUES FROM (1738368000) TO (1740787200);

CREATE TABLE ivr_cdr_2026_03 PARTITION OF ivr_cdr
    FOR VALUES FROM (1740787200) TO (1743465600);
-- ... auto-create partitions via pg_partman or cron
```

Partition pruning ensures queries for “today” only scan the current month’s partition.

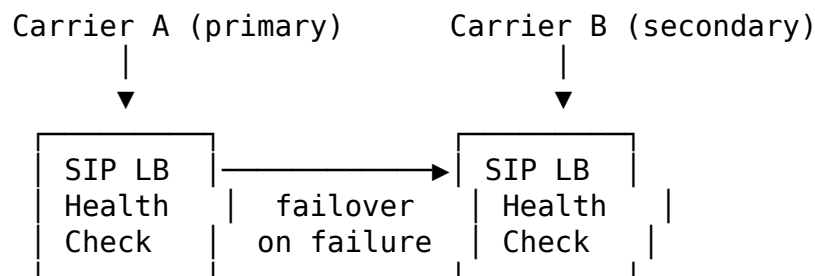
Retention: Partitions older than 90 days are detached and archived (exported to CSV on S3), then dropped.

11. Failure Handling & Resilience

11.1 FreeSWITCH Crash Recovery

Mechanism	Details
Process supervisor	systemd manages the freeswitch service with Restart=always and RestartSec=3.
Crash detection	Watchdog script (cron every 60s) verifies FreeSWITCH is responding via ESL api status. If unresponsive, force restart.
Core dumps	Enabled via ulimits; stored in /var/lib/freeswitch/core/ for post-mortem analysis.
Active call impact	Active calls are lost on crash. SIP LB detects node failure (OPTIONS ping timeout) and routes new calls to standby node within 10 seconds.
State recovery	No in-memory state persists across restarts. All IVR config is in PostgreSQL; new calls start fresh.

11.2 SIP Trunk Failure Handling



- **Primary trunk failure detection:** SIP OPTIONS pings every 10 seconds. After 3 consecutive failures (30 seconds), the trunk is marked down.
- **Failover:** Outbound calls (transfers to external numbers) switch to the secondary trunk. Inbound calls are carrier-routed; carrier-side failover to secondary DID range if configured.
- **Recovery:** When the primary trunk responds to OPTIONS again (3 consecutive successes), it is re-enabled.

11.3 Database Failover Handling

Scenario	Response
PostgreSQL primary down	Patroni promotes replica to primary within 30 seconds. PgBouncer reconnects automatically. Lua scripts experience a brief connection error; retry logic (built into <code>db_helper.lua</code>) re-establishes connection.
Both PostgreSQL nodes down	Lua scripts detect DB failure; route all calls to the operator with a system error prompt. CDR logging degrades to FreeSWITCH console log (no data loss; can be replayed).
Replica lag > 5 seconds	Dashboard queries may show stale data. Acceptable for real-time metrics (5-second refresh already tolerates this).

11.4 Dialplan Fallback Logic

If the Lua IVR script fails to execute (file missing, syntax error, runtime crash):

```
<!-- Dialplan fallback after lua execution failure -->
<extension name="ivr_fallback" continue="true">
  <condition field="variable_ivr_completed" expression="^$">
    <!-- Lua did not set ivr_completed; assume script failure -->
    <action application="playback" data="ivr/system/system_error.wav"/>
    <action application="transfer" data="operator XML ${domain_name}"/>
  </condition>
</extension>
```

This XML dialplan entry fires if the Lua script exits without setting the `ivr_completed` channel variable, catching script crashes and routing to the operator.

11.5 Retry Mechanisms Summary

Component	Retry Strategy
PostgreSQL query	1 retry, 500 ms backoff
PMS API call	1 retry, 1 second exponential backoff. 2-second timeout.
TTS API call	1 retry, 1 second backoff. Fallback to cached/pre-recorded audio.
DTMF input	Up to 3 retries (configurable) with progressive prompts
Queue timeout	Transfer to voicemail fallback handler; 1 optional re-queue
SIP trunk	Automatic failover to secondary trunk

11.6 Dead Route Handling

If a transfer destination returns SIP 404 (Not Found), 480 (Temporarily Unavailable), or 503 (Service Unavailable):

1. The `session:execute("bridge")` call returns failure.
2. The Lua script checks the hangup cause via `session:getVariable("originate_disposition")`.
3. For `NO_ROUTE_DESTINATION` or `UNALLOCATED_NUMBER`: log error, play “extension unavailable” prompt, offer voicemail or main menu.
4. For `USER_BUSY`: play “line is busy” prompt, offer retry or voicemail.
5. For `NO_ANSWER` (ring timeout): play voicemail option (Feature 2.2).

11.7 Media Server Overload Protection

Mechanism	Details
Max sessions cap	FreeSWITCH max-sessions=2000 hard limit. INVITE rejected with 503 Service Unavailable when exceeded.
Sessions per second	sessions-per-second=100 — rate limits new call setup to prevent burst overload.
CPU monitor	Lua script checks system load at call start (<code>/proc/loadavg</code>). If load average > 80% of vCPU count, play “high volume” prompt and offer callback (Phase 2) or voicemail.
SIP LB rate limiting	Kamailio/OpenSIPS rate-limits INVITE by source IP: max 50 INVITE/second per carrier.

12. Observability & Monitoring

12.1 FreeSWITCH Logs

Log Type	Location	Details
Console log	<code>/var/log/freeswitch/freeswitch.log</code>	All FreeSWITCH core and module logs. Level: WARNING in production.
SIP trace	<code>/var/log/freeswitch/sip_trace.log</code>	Enabled on-demand for debugging (sofia global siptrace on).
CDR XML	<code>/var/log/freeswitch/xml_cdr.xml</code>	Raw XML CDR files (retained 7 days on disk).

Log Type	Location	Details
IVR Lua logs	Within freeswitch.log	Prefixed with [IVR] for grep filtering.

Log rotation: logrotate configured to rotate freeswitch.log daily, compress after 1 day, retain 30 days.

12.2 FusionPBX Logs

Log	Location	Details
Application log	/var/log/fusionpbx/app.log	PHP application errors and admin actions
Access log	Nginx access log	HTTP request log for web UI and API
Auth log	/var/log/fusionpbx/login_failed.log	Failed login attempts (Fail2Ban source)

12.3 PostgreSQL Logs

```
# postgresql.conf logging configuration
log_destination = 'stderr'
logging_collector = on
log_directory = '/var/log/postgresql'
log_filename = 'postgresql-%Y-%m-%d.log'
log_rotation_age = 1d
log_rotation_size = 100MB
log_min_duration_statement = 200      # Log queries > 200ms
log_checkpoints = on
log_connections = on
log_disconnections = on
log_lock_waits = on
log_statement = 'ddl'                # Log DDL statements
```

12.4 Call Metrics Monitoring

Metric	Collection Method	Alert Threshold
Active concurrent calls	ESL show calls count	> 80% of max-sessions
Calls per second	ESL status	> 80 CPS (of 100 max)
Average call setup time	IVR CDR (answer_epoch - start_epoch)	> 3 seconds

Metric	Collection Method	Alert Threshold
IVR abandonment rate	IVR CDR aggregation	> 15% over 30 min
Queue wait time	IVR CDR queue_time_seconds	> 120 seconds average
PMS API latency	Lua script timing logs	> 500 ms P95
TTS API latency	Lua script timing logs	> 1000 ms P95
Database query time	PostgreSQL slow query log	> 200 ms

12.5 SIP Health Checks

Check	Method	Frequency
FreeSWITCH alive	ESL api status	Every 30 seconds (local watchdog)
SIP profile active	sofia status profile external	Every 60 seconds
Trunk registration	sofia status gateway {gw_name}	Every 60 seconds
End-to-end SIP test	Synthetic SIP OPTIONS from monitoring node	Every 5 minutes

12.6 Alerting Strategy

Alert Level	Channel	Response
Critical	PagerDuty / SMS	FreeSWITCH down, DB down, all trunks down. Immediate response.
Warning	Email + Slack	High abandonment rate, queue overflow, PMS API errors. Response within 30 min.
Info	Slack / Dashboard	Call volume spike, TTS cache miss rate high. Awareness only.

Alerting is implemented via: - **Node health:** Prometheus node_exporter + Grafana alerting (or a lightweight cron-based checker for MVP). - **Call metrics:** Dashboard alert indicators (Feature 5.2) + email notifications from a scheduled PostgreSQL query.

12.7 Log Retention Policy

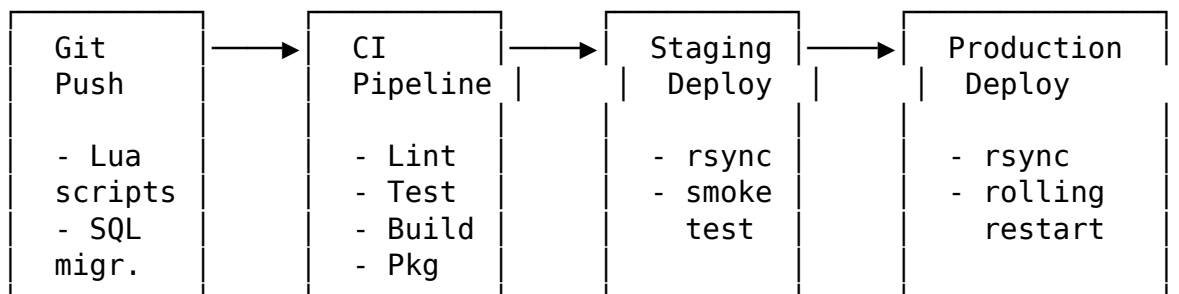
Log Type	Retention	Archive
FreeSWITCH console log	30 days on disk	Compressed to S3 after 7 days
SIP trace log	3 days on disk	On-demand capture only
XML CDR files	7 days on disk	Ingested to DB; files deleted
PostgreSQL query log	30 days on disk	Compressed to S3 after 7 days
IVR CDR (database)	90 days live	Archived to S3/CSV after 90 days
FusionPBX app log	30 days	Compressed to S3 after 7 days
Audit log (database)	1 year live	Archived after 1 year

13. Deployment & DevOps Strategy

13.1 Environment Setup

Environment	Purpose	Infrastructure
Development	Developer workstations; local FusionPBX VM for Lua script testing.	Single VM (Vagrant/Docker), local PostgreSQL, mock PMS API.
Staging	Pre-production validation; mirrors production topology.	2 FusionPBX nodes, 1 PostgreSQL (no HA), staging SIP trunk, test PMS API.
Production	Live traffic.	Full HA cluster per Section 2.2.

13.2 CI/CD for Lua Scripts

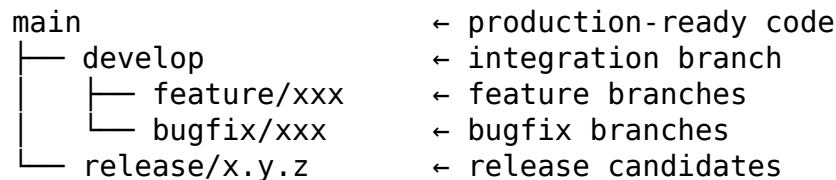


CI Pipeline Steps

1. **Lint:** luacheck for Lua syntax and style validation.
2. **Unit Test:** Lua test suite (busted framework) runs against mock session/DB objects.
3. **SQL Migration Validation:** Dry-run migrations against a test PostgreSQL instance.
4. **Package:** Bundle Lua scripts and migrations into a versioned tarball.
5. **Deploy to Staging:** rsync scripts to staging FusionPBX node; run migrations; execute smoke tests (synthetic SIP call through the IVR).
6. **Manual Approval Gate:** Staging sign-off before production.
7. **Deploy to Production:** rsync to production nodes (Node A first, then Node B); run migrations on primary PostgreSQL.

13.3 Version Control Strategy

Repository: jazzms-ivr-auto-attendant



- All Lua scripts, SQL migrations, dialplan XML templates, and configuration files are version-controlled.
- Audio prompt files are NOT stored in Git (too large). They are managed via the FusionPBX admin UI and backed up to S3.
- Deployment is triggered by merging to main and tagging a release.

13.4 Blue-Green Deployment

The two-node FusionPBX cluster enables a rolling deployment strategy:

1. **Drain Node B:** SIP LB stops sending new calls to Node B. Existing calls complete naturally (or are drained with a 60-second timeout).
2. **Deploy to Node B:** rsync new Lua scripts; run schema migrations (if any).
3. **Test Node B:** Synthetic SIP test call to Node B directly.
4. **Activate Node B:** SIP LB resumes routing to Node B.
5. **Drain Node A:** Same process.
6. **Deploy to Node A.**
7. **Activate Node A.**

Zero-downtime deployment: the SIP LB always has at least one active node.

13.5 Database Migration Handling

1. Migrations are **forward-only** (no down migrations in MVP). Rollback is handled by deploying a corrective migration.

2. Migrations run on the PostgreSQL primary. Replica picks up changes via streaming replication.
3. Migrations that add columns use ADD COLUMN ... DEFAULT ... NOT NULL (PostgreSQL 11+ does this without table rewrite).
4. Migrations that add tables are non-disruptive.
5. Destructive migrations (DROP COLUMN, DROP TABLE) require manual approval and a maintenance window.
6. Migration runner:

```
#!/bin/bash
for migration in migrations/*.sql; do
    version=$(basename "$migration" | cut -d_ -f1)
    applied=$(psql -t -c "SELECT COUNT(*) FROM ivr_schema_version WHERE version=$version")
    if [ "$applied" -eq 0 ]; then
        psql -f "$migration" && \
        psql -c "INSERT INTO ivr_schema_version (version, description) VALUES ($version, '$migration')"
    fi
done
```

13.6 Backup & Restore Plan

Scenario	Recovery Procedure	RTO	RPO
Single FusionPBX node failure	SIP LB routes to standby node. No action needed.	< 10 seconds	0 (no data loss)
PostgreSQL primary failure	Patroni auto-promotes replica. PgBouncer reconnects.	< 30 seconds	< 1 second
Both FusionPBX nodes down	Provision new node from image; rsync Lua scripts from Git; point to DB.	< 30 minutes	0 (DB intact)
PostgreSQL both nodes down	Restore from S3 pg_basebackup + WAL replay.	< 1 hour	< 1 second (WAL)
Complete datacenter failure	Provision in new region from S3 backups + Git.	< 4 hours	< 1 hour (last backup)
Lua script rollback	git revert + redeploy via CI/CD.	< 15 minutes	N/A
Audio prompt rollback	Restore from S3 backup; update ivr_prompts version.	< 15 minutes	< 1 hour

14. Technology Stack

14.1 Fixed Requirements

Component	Technology	Version	Justification
Telephony Platform	FusionPBX	Latest stable (5.x)	Multi-tenant PBX management with web UI, dialplan management, voicemail, ring groups, and PostgreSQL backend. Provides the operational framework for the IVR.
Media Server / Call Control	FreeSWITCH	Latest stable (1.10.x)	High-performance, carrier-grade SIP B2BUA and media server. Handles all SIP signaling, RTP/SRTP media, DTMF processing, codec transcoding, and call bridging. Proven at 10,000+ concurrent call deployments.
Database	PostgreSQL	15+ (FusionPBX default)	ACID-compliant relational database with JSONB support for flexible CDR data, declarative partitioning for CDR table management, and streaming replication for HA. Already embedded in FusionPBX.
IVR Scripting	Lua	5.2 (FreeSWITCH embedded)	Lightweight, fast scripting language embedded in FreeSWITCH via mod_lua. Sub-millisecond startup, low memory footprint, and direct access to FreeSWITCH session API.
Operating System	Ubuntu	22.04 LTS	Long-term support (until 2027). Excellent FreeSWITCH/FusionPBX package availability. Familiar operations toolchain.
SIP Load Balancer	Kamailio	5.7+	SIP-aware proxy with high performance (50,000+ TPS), health checking, failover, and rate limiting. Production-proven in carrier environments.

Component	Technology	Version	Justification
Connection Pooler	PgBouncer	1.21+	Lightweight PostgreSQL connection pooler. Transaction-mode pooling supports hundreds of Lua concurrent sessions with a fraction of actual database connections.
HA Orchestrator	Patroni	3.x	PostgreSQL HA framework with automatic failover, leader election via etcd/consul, and REST API for health monitoring.
TTS Engine	Google Cloud TTS	v1 API	Best-in-class voice quality (WaveNet/Neural2 voices). 500 ms P95 latency. Pay-per-use pricing aligned with IVR traffic patterns. AWS Polly as secondary option.
Monitoring	Prometheus + Grafana	Latest	Industry-standard metrics collection and dashboarding. node_exporter for system metrics. Custom exporter for FreeSWITCH/IVR metrics.

14.2 Configuration Decisions and Tuning Parameters

FreeSWITCH Core Configuration

Parameter	Value	Rationale
max-sessions	2000	4x headroom over expected 500 concurrent calls. Prevents unbounded resource consumption.
sessions-per-second	100	Prevents call-setup storms. 100 CPS handles peak hotel traffic with margin.
rtp-start-port	10000	Standard RTP port range start.
rtp-end-port	20000	10,000 port range supports 5,000 concurrent RTP streams (2 ports per call).

Parameter	Value	Rationale
max-db-handles	50	FreeSWITCH internal DB connection pool. Aligned with PgBouncer pool size.
core-db-dsn	PgBouncer endpoint	All FreeSWITCH core DB operations go through PgBouncer for connection management.

mod_lua Configuration

Parameter	Value	Rationale
xml-handler-script	Not used	IVR logic is invoked via dialplan application="lua", not XML handler, for simplicity.
startup-script	Not used	No long-running Lua processes. Each call spawns a fresh Lua instance.
Script path	/usr/share/freeswitch/standard-fusionpbx script directory.	Standard FusionPBX script directory.

mod_sofia SIP Profile (External)

Parameter	Value	Rationale
sip-port	0 (disabled)	No unencrypted SIP in production.
tls-sip-port	5061	Standard SIP-TLS port.
rtp-secure-media	mandatory	All media encrypted per Feature 6.1.
codec-prefs	PCMU,PCMA,G729	G.711 for quality and compatibility; G.729 for bandwidth-constrained trunks.
inbound-reg-force-matching-username	true	Prevents SIP registration spoofing.
auth-calls	true	All inbound calls authenticated (IP ACL or digest).
rtp-timeout-sec	300	Detect dead calls (no RTP for 5 min → hangup).
rtp-hold-timeout-sec	1800	Calls on hold for 30 min → hangup (prevent ghost calls).

PostgreSQL Tuning

Parameter	Value	Rationale
shared_buffers	2 GB	25% of 8 GB RAM. Standard recommendation for database-heavy workloads.
effective_cache_size	6 GB	75% of RAM. Informs planner about OS cache availability.
work_mem	64 MB	Adequate for CDR aggregation queries with sorting.
max_connections	200	Behind PgBouncer; actual concurrent connections ~50.
checkpoint_completion_target	0.9	Spread checkpoint I/O over 90% of the interval for smooth performance.
wal_level	replica	Required for streaming replication.
max_wal_senders	5	Supports replica + backup connections.
synchronous_commit	on	Durability guarantee for CDR writes.

Appendix A: Glossary

Term	Definition
ANI	Automatic Number Identification — the caller's phone number
B2BUA	Back-to-Back User Agent — SIP entity that terminates and re-originates calls
CDR	Call Detail Record
DID	Direct Inward Dialing number
DNIS	Dialed Number Identification Service — the number the caller dialed
DND	Do Not Disturb
DTMF	Dual-Tone Multi-Frequency — touch-tone keypad signals
ESL	Event Socket Library — FreeSWITCH programmatic control interface
FIFO	First In, First Out — call queue mechanism in FreeSWITCH
MWI	Message Waiting Indicator — visual/audible notification on IP phones
PMS	Property Management System — hotel operations software
RTP	Real-time Transport Protocol — media transport

Term	Definition
SDES	Session Description Protocol Security Descriptions — SRTP key exchange method
SDP	Session Description Protocol — media negotiation in SIP
SRTP	Secure Real-time Transport Protocol — encrypted media
TLS	Transport Layer Security — encrypted signaling transport
TTS	Text-to-Speech
VRRP	Virtual Router Redundancy Protocol — IP failover mechanism
WAL	Write-Ahead Log — PostgreSQL transaction log for replication and recovery

Appendix B: MVP Feature-to-Section Traceability Matrix

Feature ID	Feature Name	Sections Covered
1.1	Multi-Level IVR Navigation	3.1, 3.3, 3.10, 4.2, 4.3, 5.3.1, 5.3.2, 7.1
1.2	DTMF Input Capture & Validation	3.5, 4.3, 2.1.2
1.3	TTS Integration	4.4, 6.2, 6.3
1.4	Audio File Upload & Management	4.4, 6.1, 6.4, 6.6
1.5	Configurable Timeouts & Retries	3.6, 4.3, 5.3.8
1.6	No-Input Timeout Handling	3.7, 4.3, 4.5
1.7	Invalid Input Handling	3.7, 4.3
2.1	Time-Based Routing	3.9, 5.3.4, 7.1, 7.2
2.2	Voicemail Routing & Fallback	3.8.3, 7.4, 7.5, 7.6
2.3	Room Extension Routing	3.10, 4.1 (script structure), 5.3.3, 7.1
3.1	Business Hours Calendar	3.9, 5.3.4, 7.1
3.2	Language Selection (English MVP)	4.2, 6.7
4.1	PMS Integration	3.10, 4.7, 11.5
4.2	Hotel PBX Integration	2.1.2, 3.2, 3.8, 9.1
5.1	Call Detail Records (CDR)	5.3.6, 8.1, 8.3, 10.7
5.2	Real-Time Metrics Dashboard	5.3.7, 8.2, 8.4
6.1	Encryption (TLS + SRTP)	9.1